BPS5231

Artificial Intelligence

For

Sustainable Building Design

L7

Ang Yu Qian

Assistant Professor

Announcement

Field Trip (Finally)

15 Oct 2025 (Wed)

Bus leaves 1.55pm from SDE3 Foyer

L07.1
Supervised 6

Neural Networks

=

L07.2
Simulations

Rhino

ClimateStudio

Grasshopper

L07.1

Supervised 6

Neural Networks

=

L07.2

Simulations

Rhino

ClimateStudio

Grasshopper

Today, we will cover

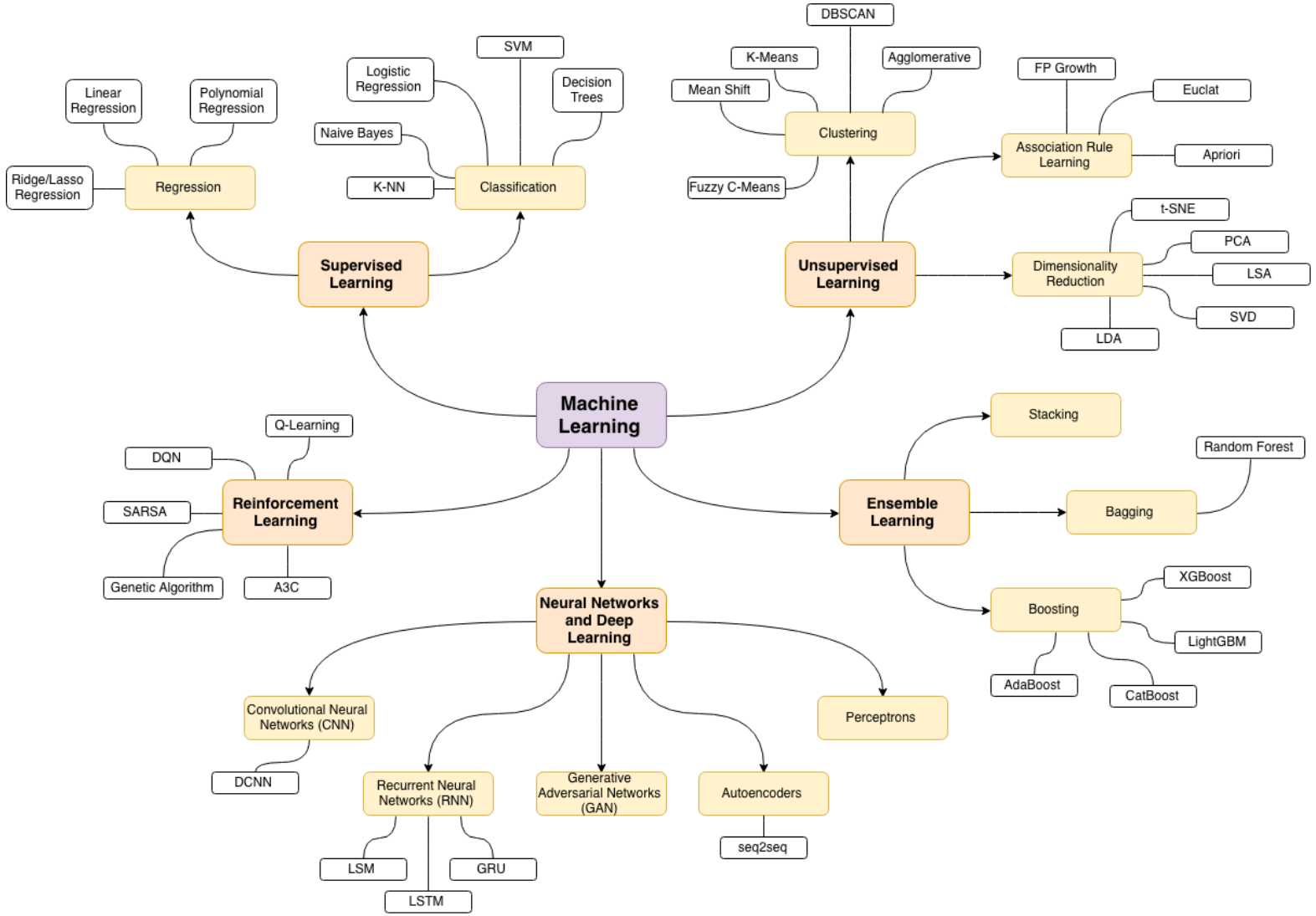


Image from Anil Jain, Google

Today, we will cover

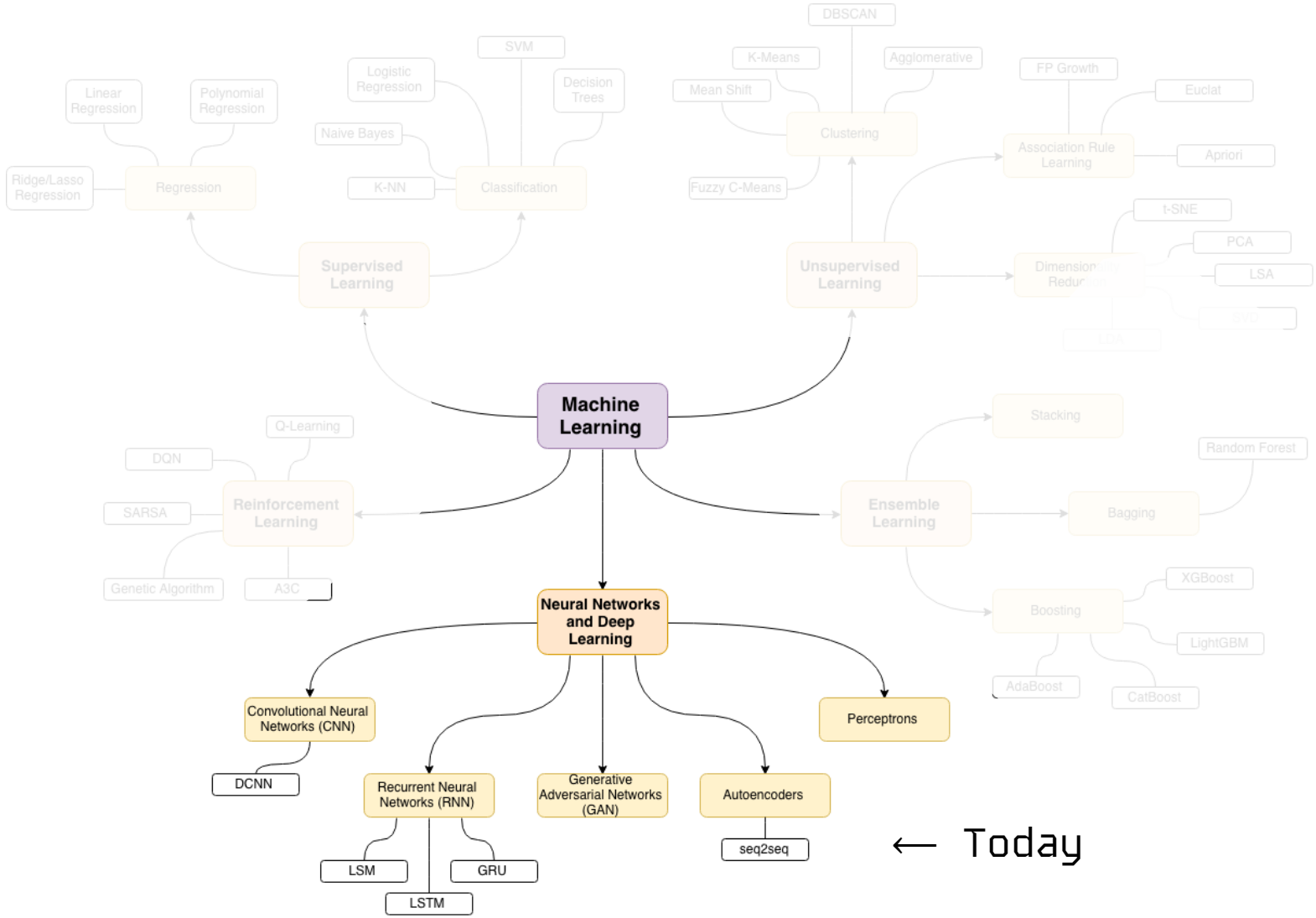


Image from Anil Jain, Google

Recap – Bagging

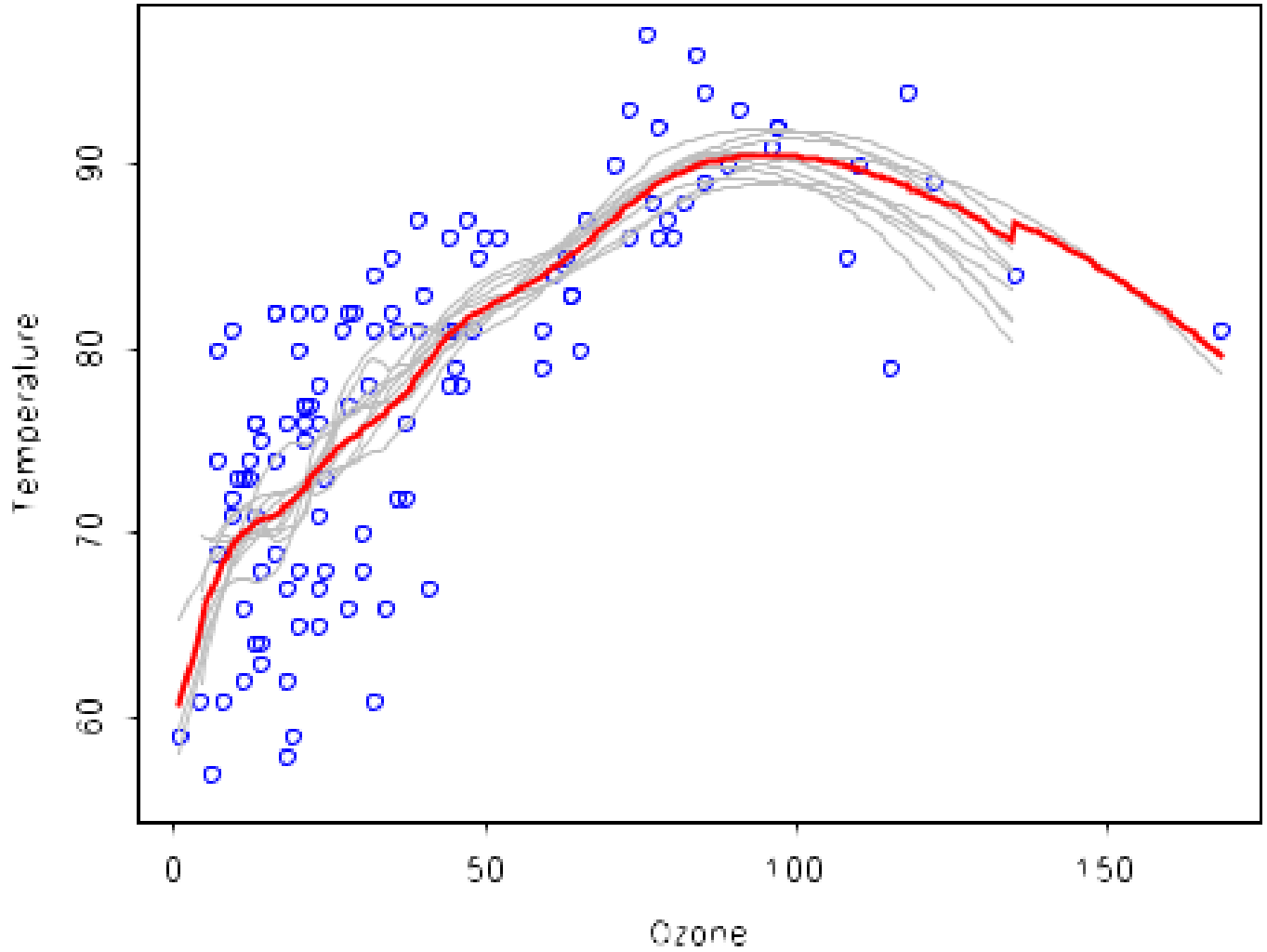
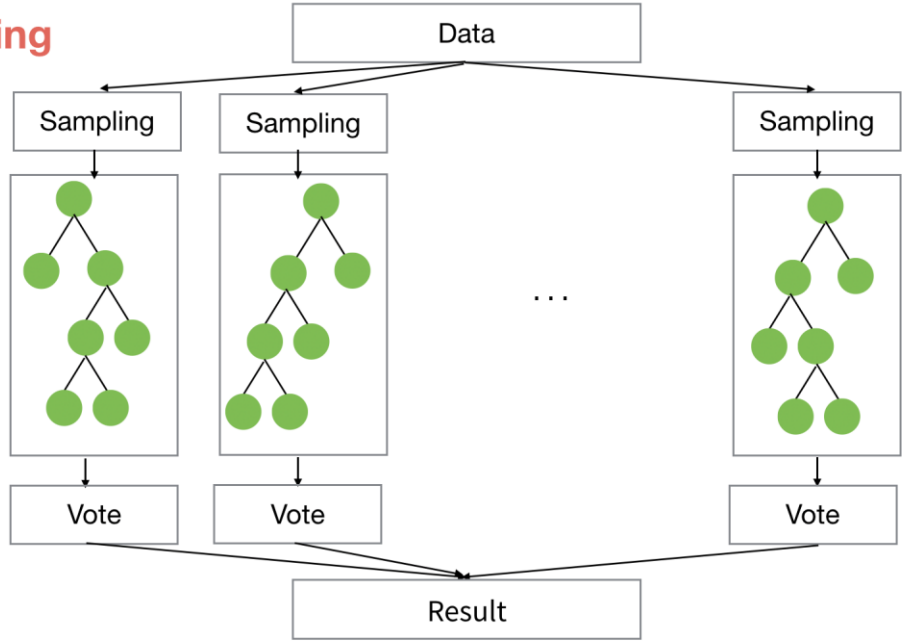


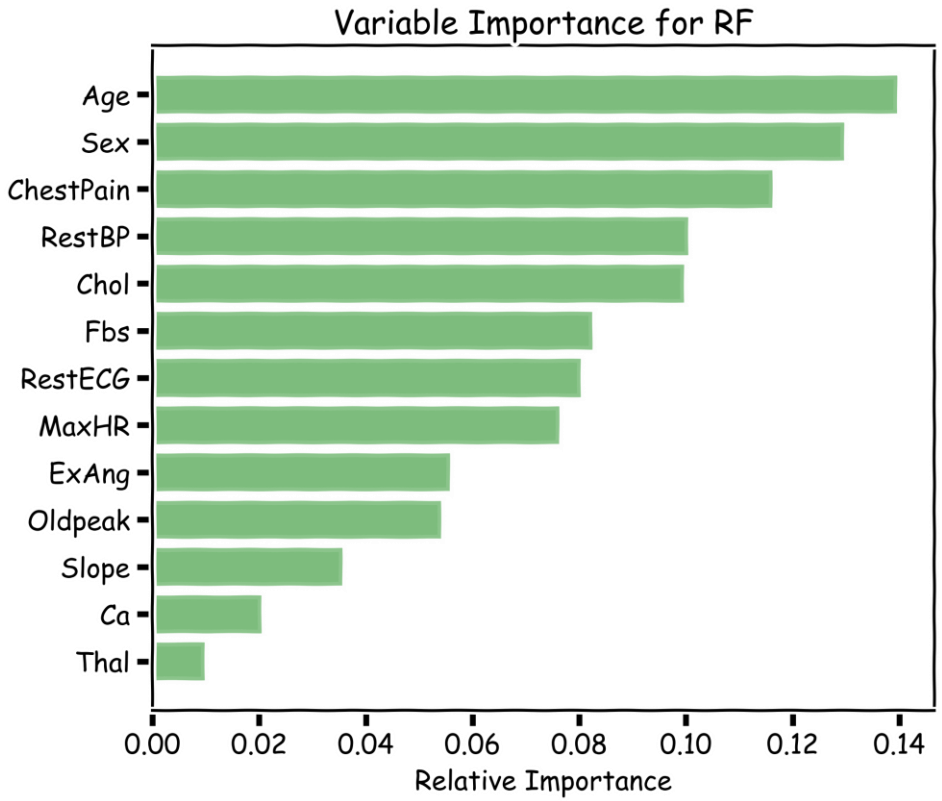
Image from Palvos Protopapas

Recap – Ensemble Models

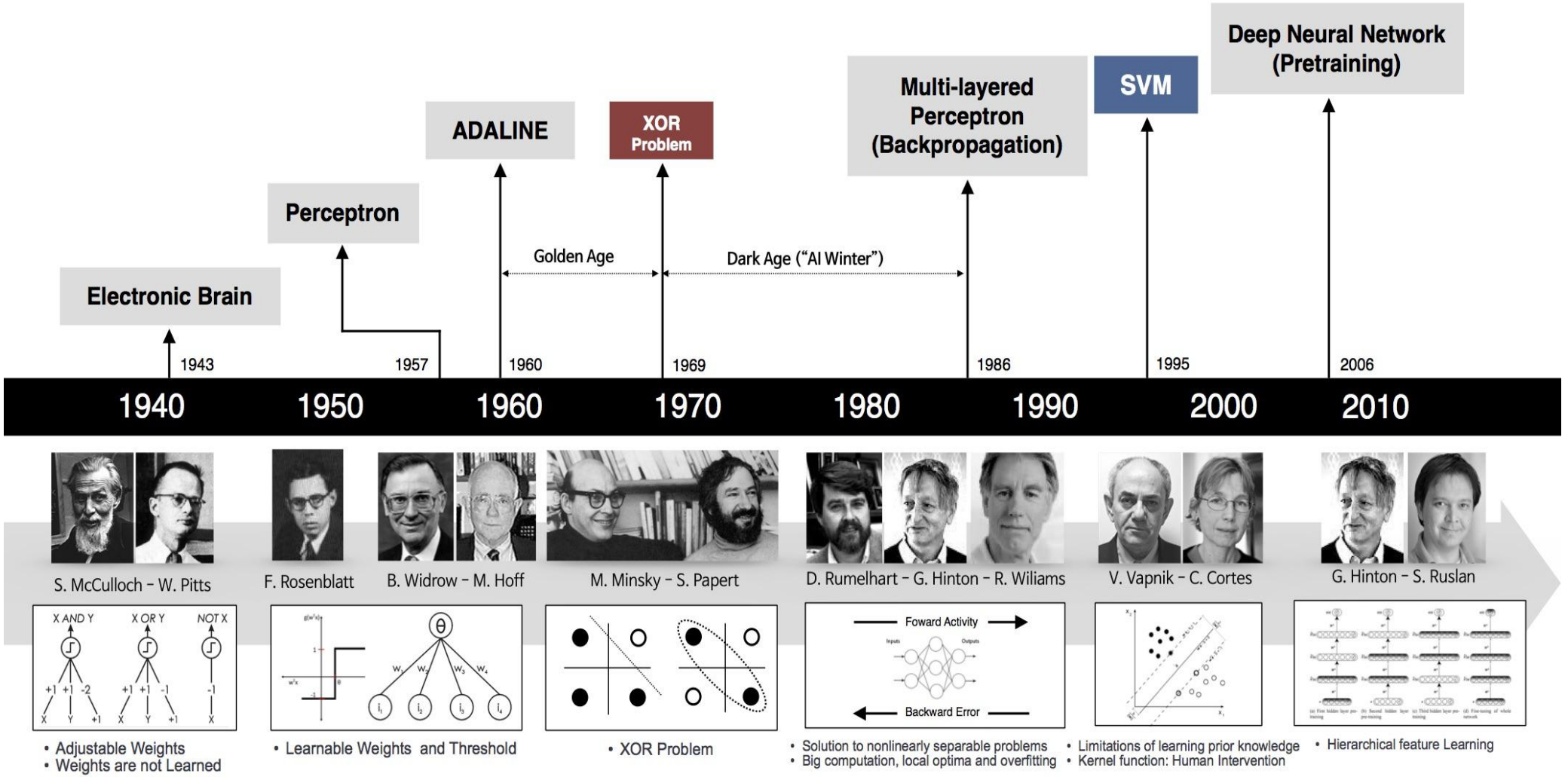
Random Sampling

Decision Tree





Neural Networks



beamandrew.github.io/

Recap – Regression and Classification

Methods that are centered around modeling and prediction of a quantitative response variable (e.g. building EUI, number of faults, etc) are called regressions (and Ridge, LASSO, etc).

When the response variable is categorical, then the problem is instead labeled as a classification problem.

The goal is to attempt to classify each observation into a category (e.g. building type) defined by Y, based on a set of predictor variables X.

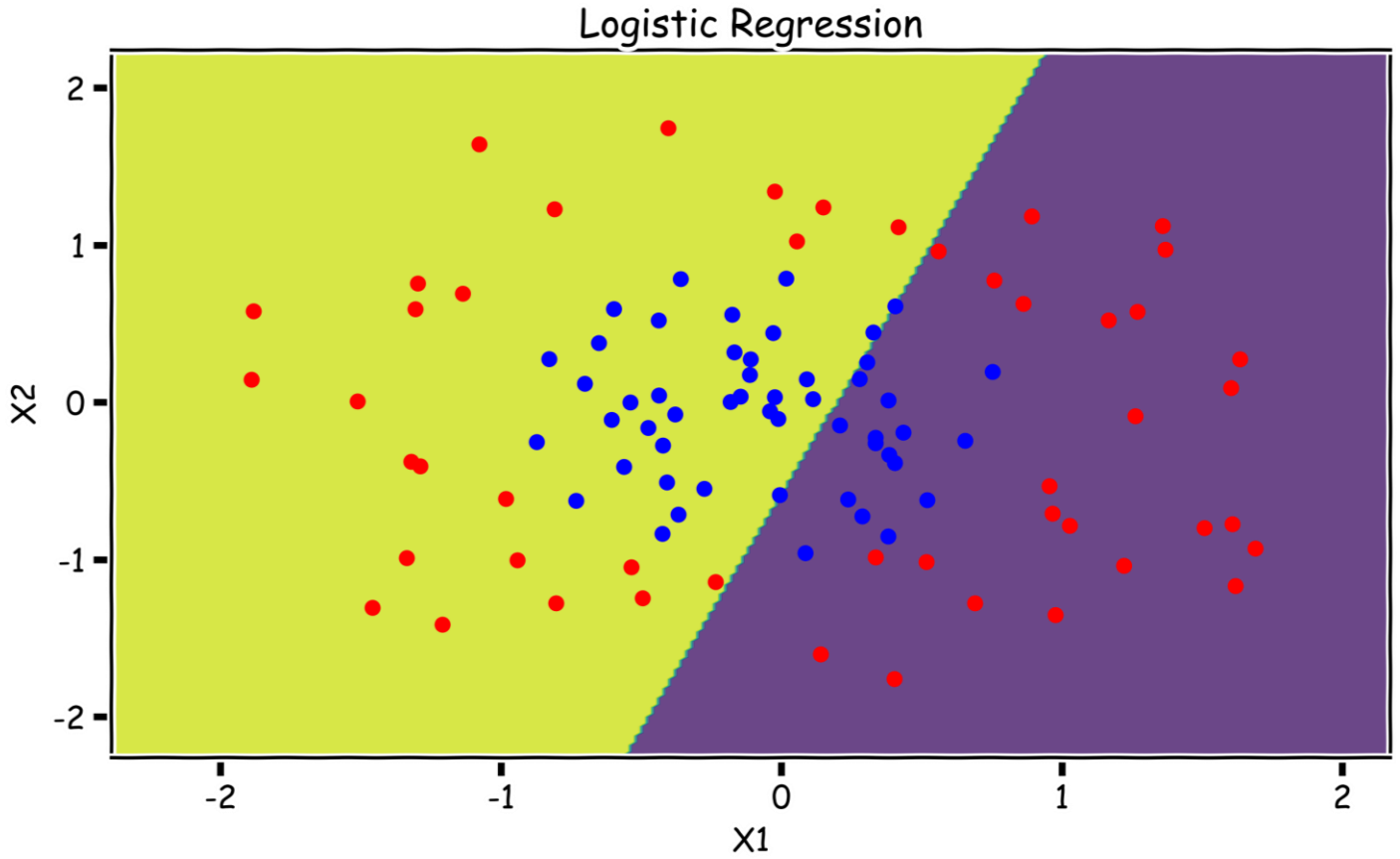
Recap – Regression and Classification

Methods that are centered around modeling and prediction of a quantitative response variable (e.g. building EUI, number of faults, etc) are called regressions (and Ridge, LASSO, etc).

When the response variable is categorical, then the problem is instead labeled as a classification problem.

The goal is to attempt to classify each observation into a category (e.g. building type) defined by Y, based on a set of predictor variables X.

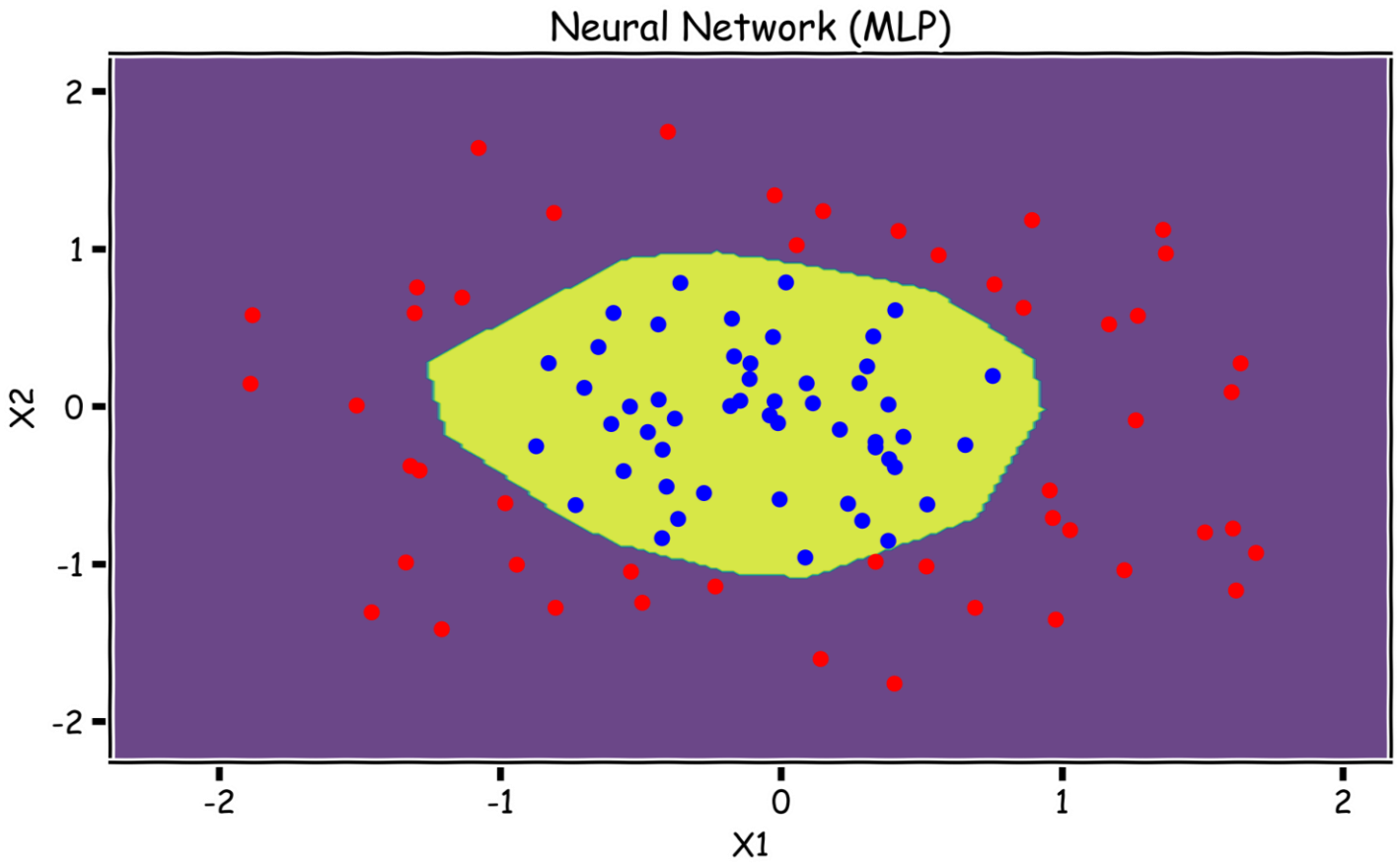
Logistic Regression Example



Without augmenting the features (i.e. without adding $X1^2$ or $X2^2$ non-linear features), Logistic Regression is incapable of modeling the correct decision boundary.

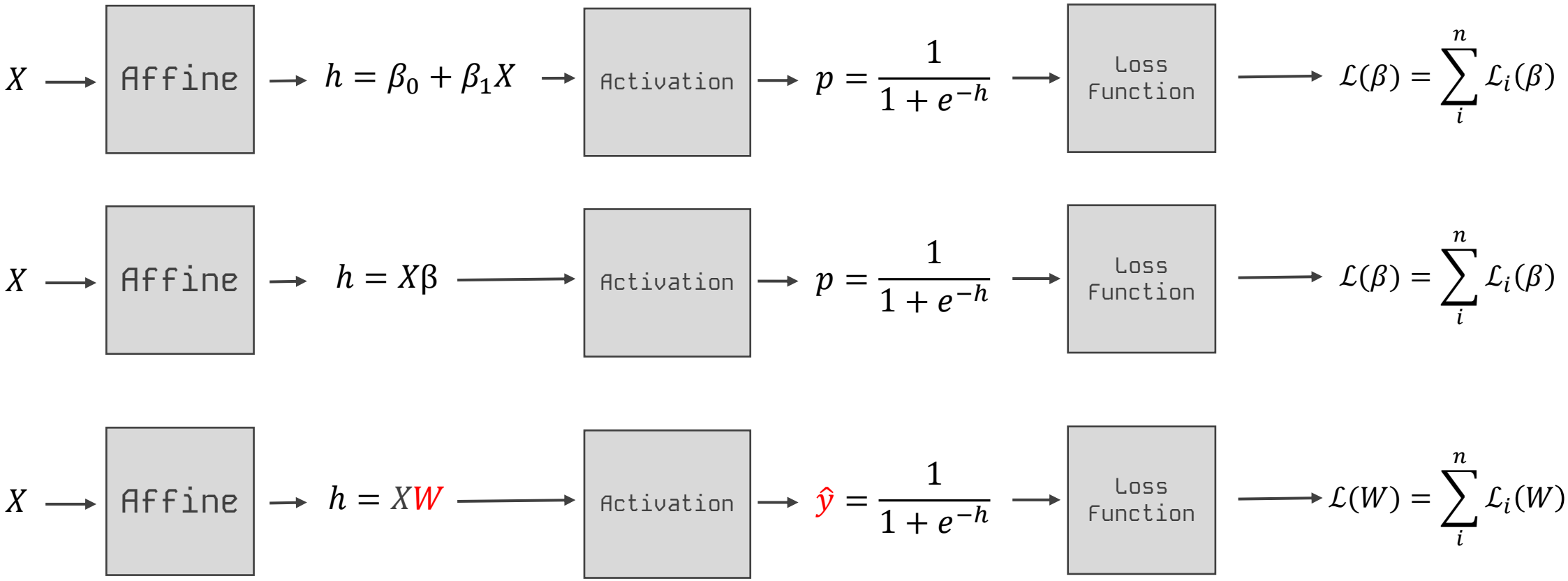
Image from Palvos Protopapas

Neural Networks

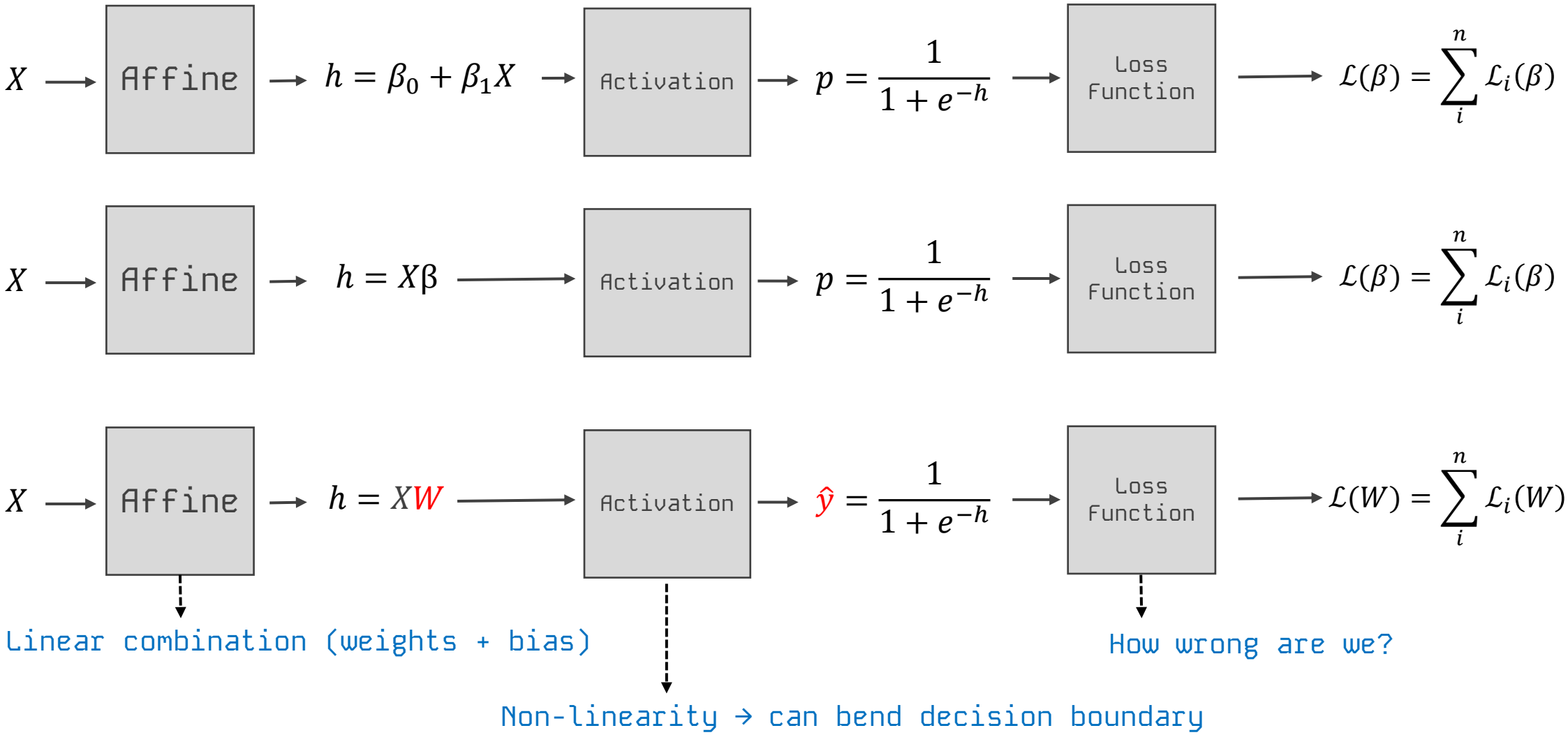


A neural network is a powerful non-linear model that can easily model the non-linear decision boundary correctly.

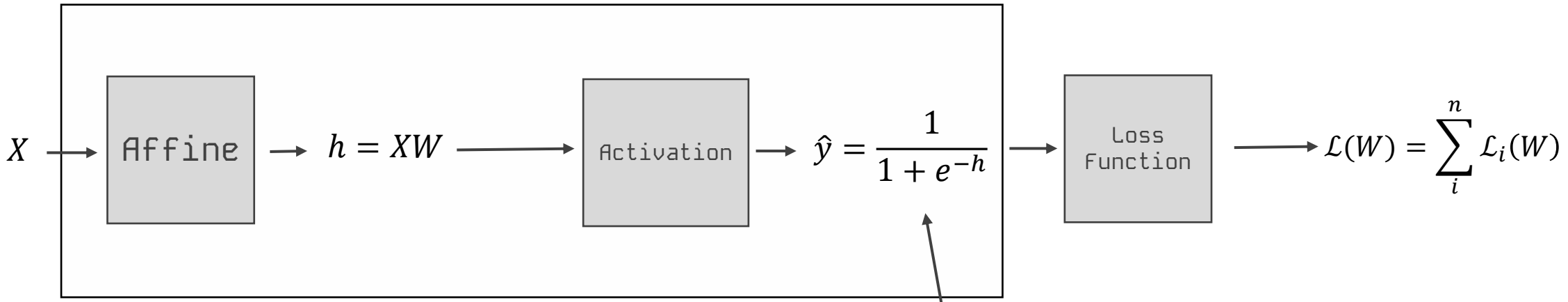
Image from Palvos Protopapas


X
Affine
$h = \beta_0 + \beta_1 X$
Activation
$p = \frac{1}{1+e^{-h}}$
Loss Function
$\mathcal{L}(\beta) = \sum_i^n \mathcal{L}_i(\beta)$
X
Affine
$h = X\beta$
Activation
$p = \frac{1}{1+e^{-h}}$
Loss Function
$\mathcal{L}(\beta) = \sum_i^n \mathcal{L}_i(\beta)$
X
Affine
$h = XW$
Activation
$\hat{y} = \frac{1}{1+e^{-h}}$
Loss Function
$\mathcal{L}(W) = \sum_i^n \mathcal{L}_i(W)$

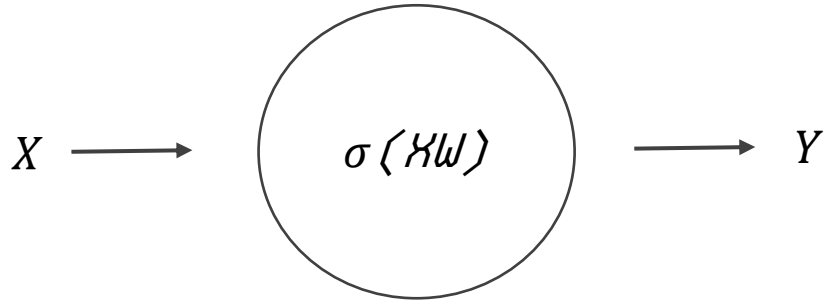
Neural Networks

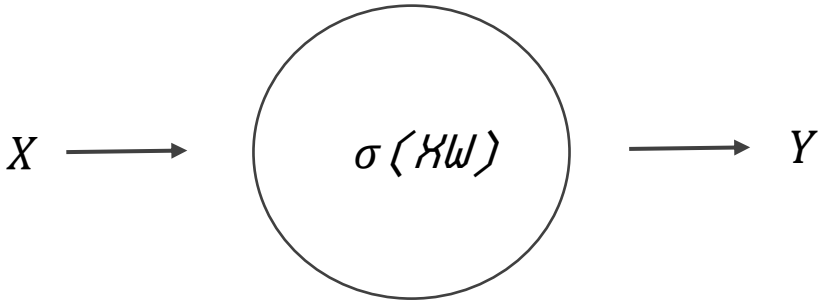


Neural Networks



"Sigmoid activation" σ





Choose W such as

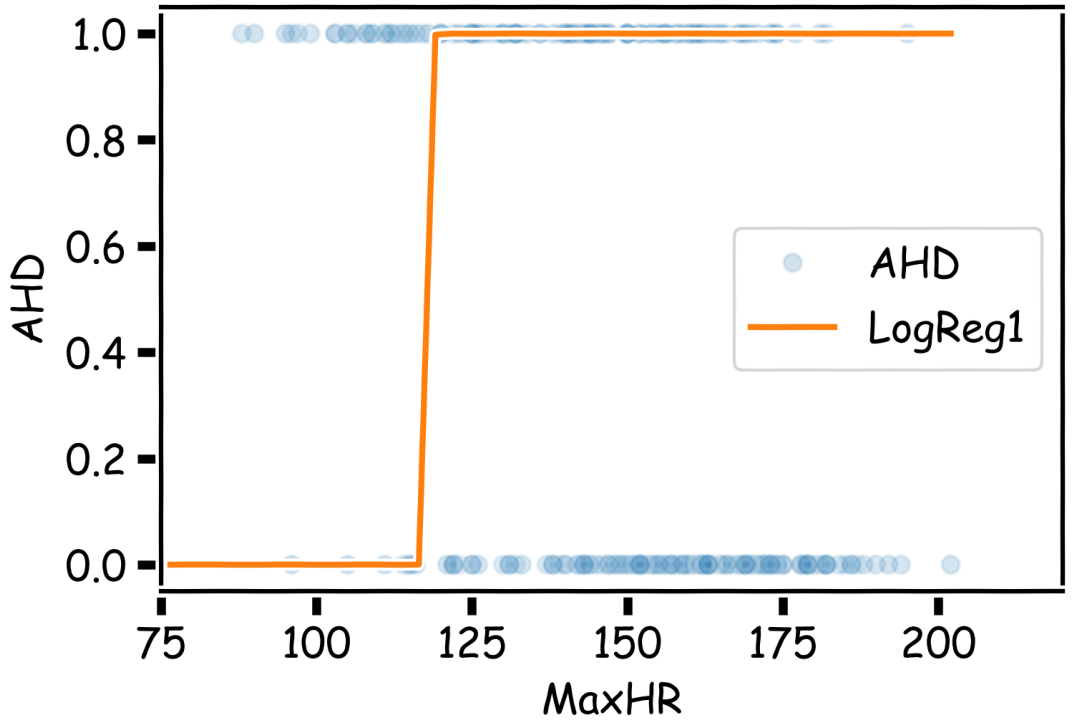
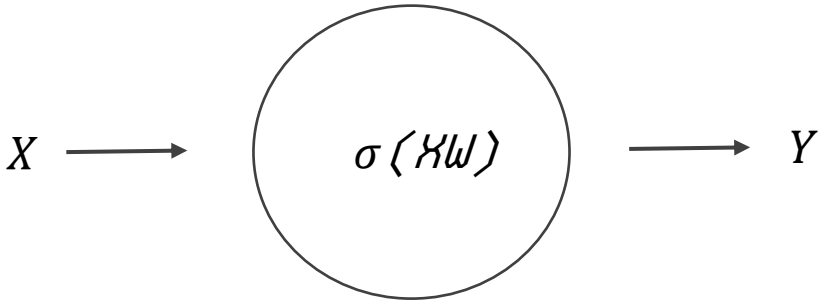


Image from Palvos Protopapas



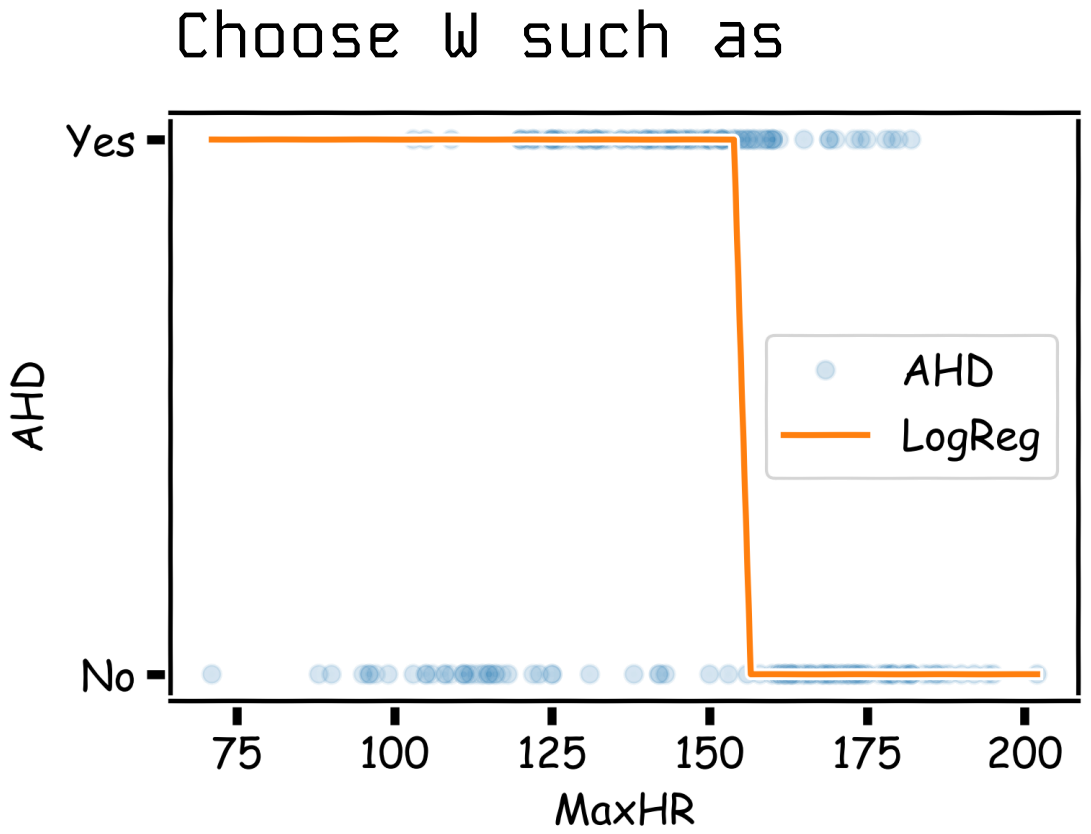
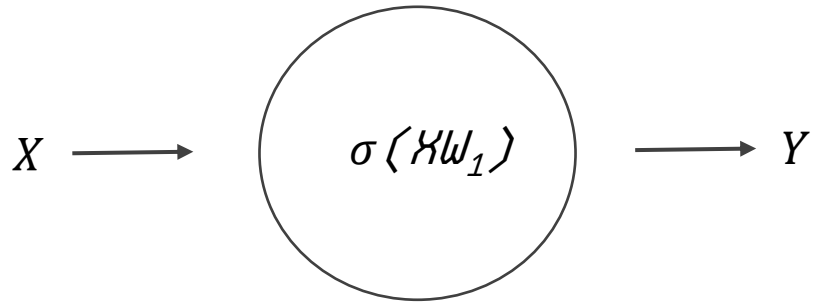
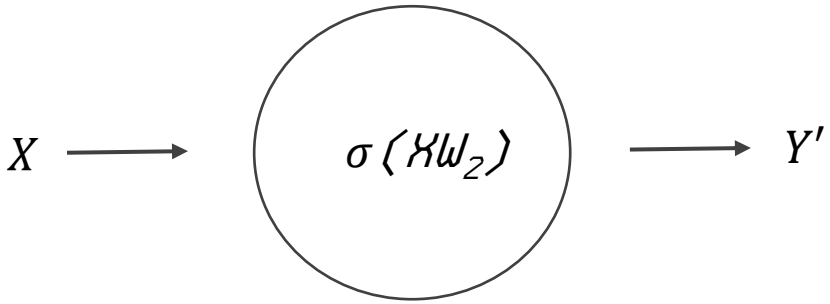
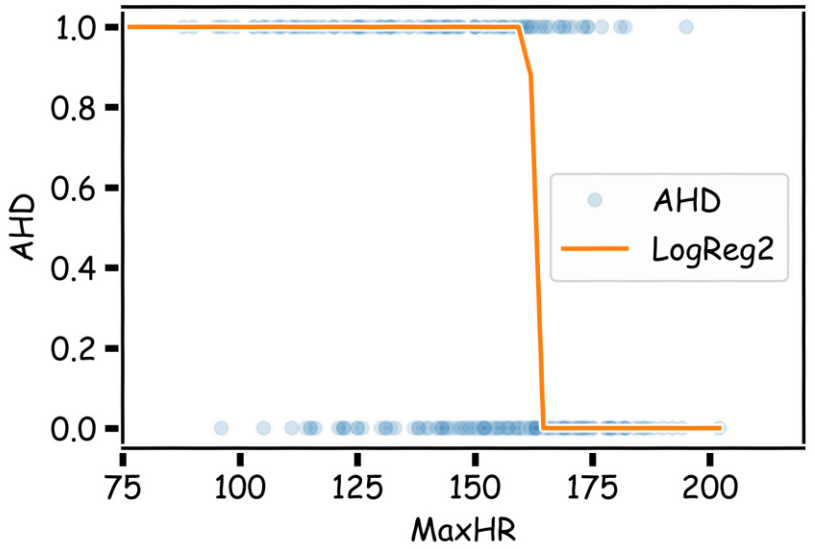


Image from Palvos Protopapas

Neural Networks







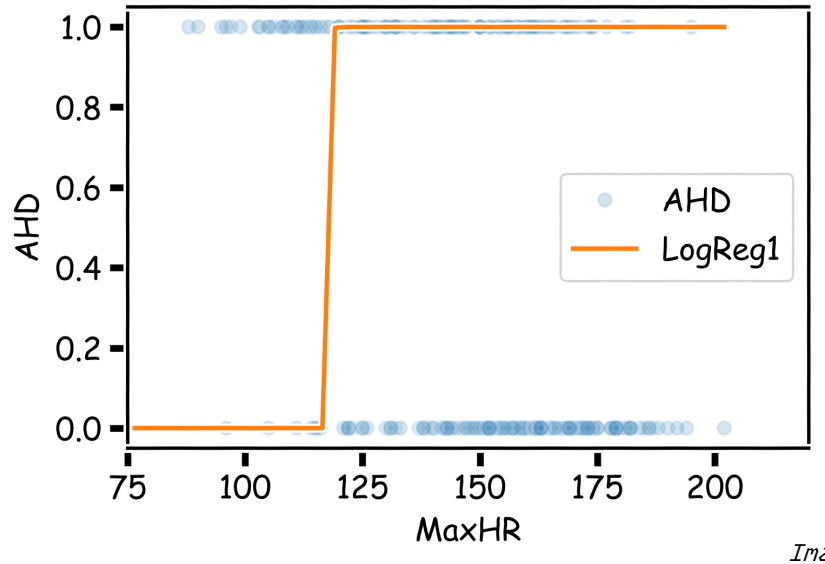


Image from Palvos Protopapas

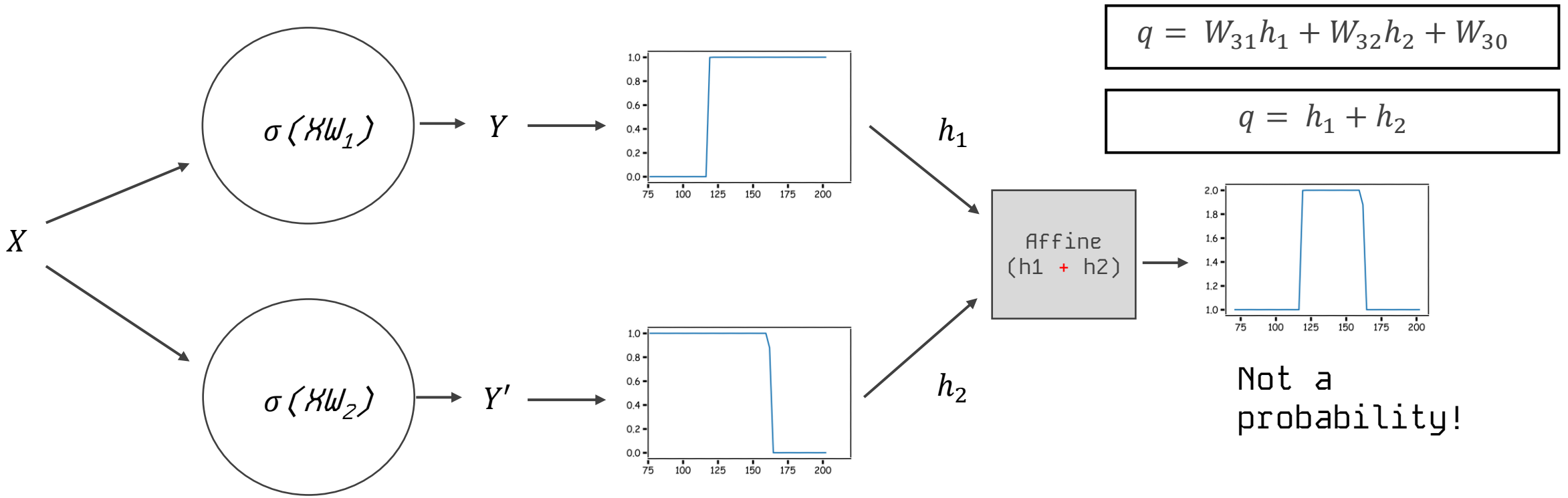


Image from Palvos Protopapas

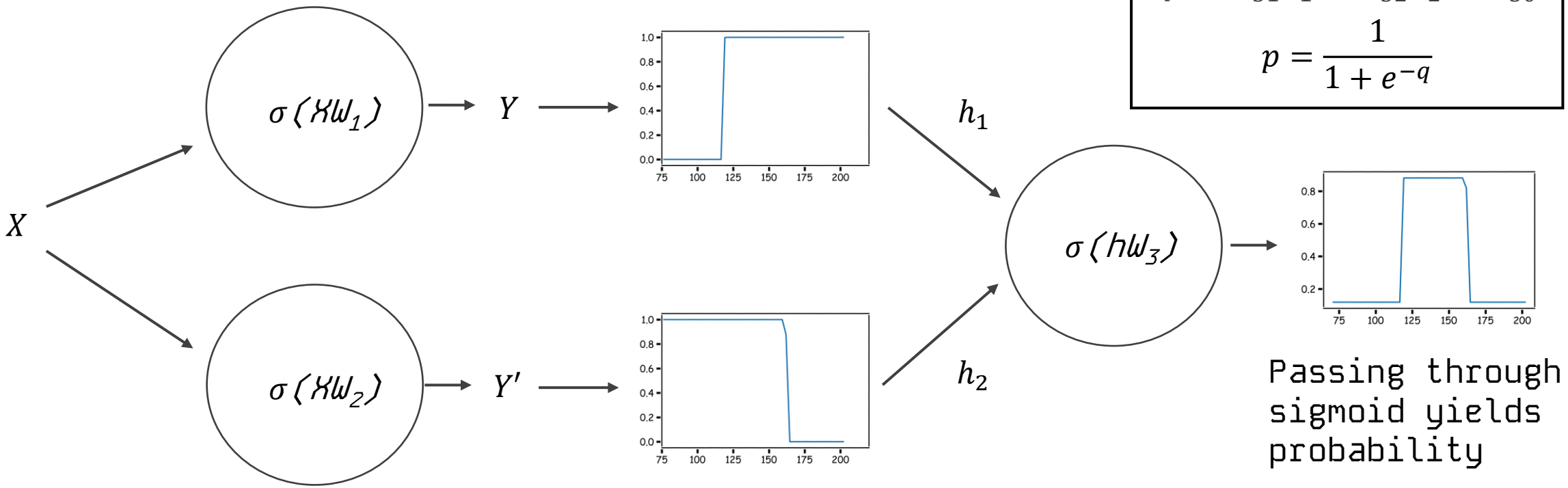


Image from Palvos Protopapas

Combining neurons allows us to model interesting functions

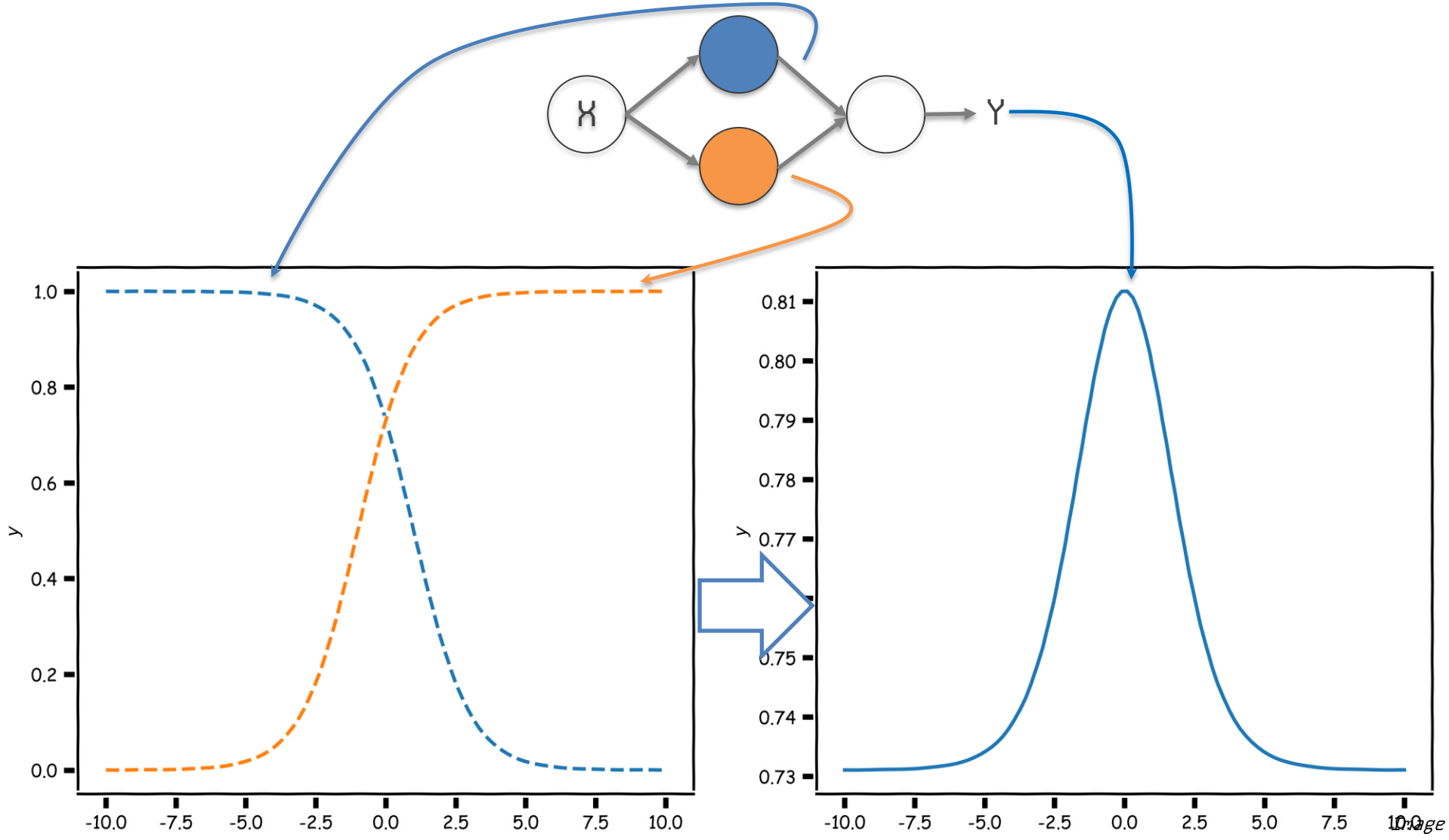


Image from Palvos Protopapas

SUPERVISE 6 SUPERVISE 7 SIMULATIONS

Different weights change the shape and position

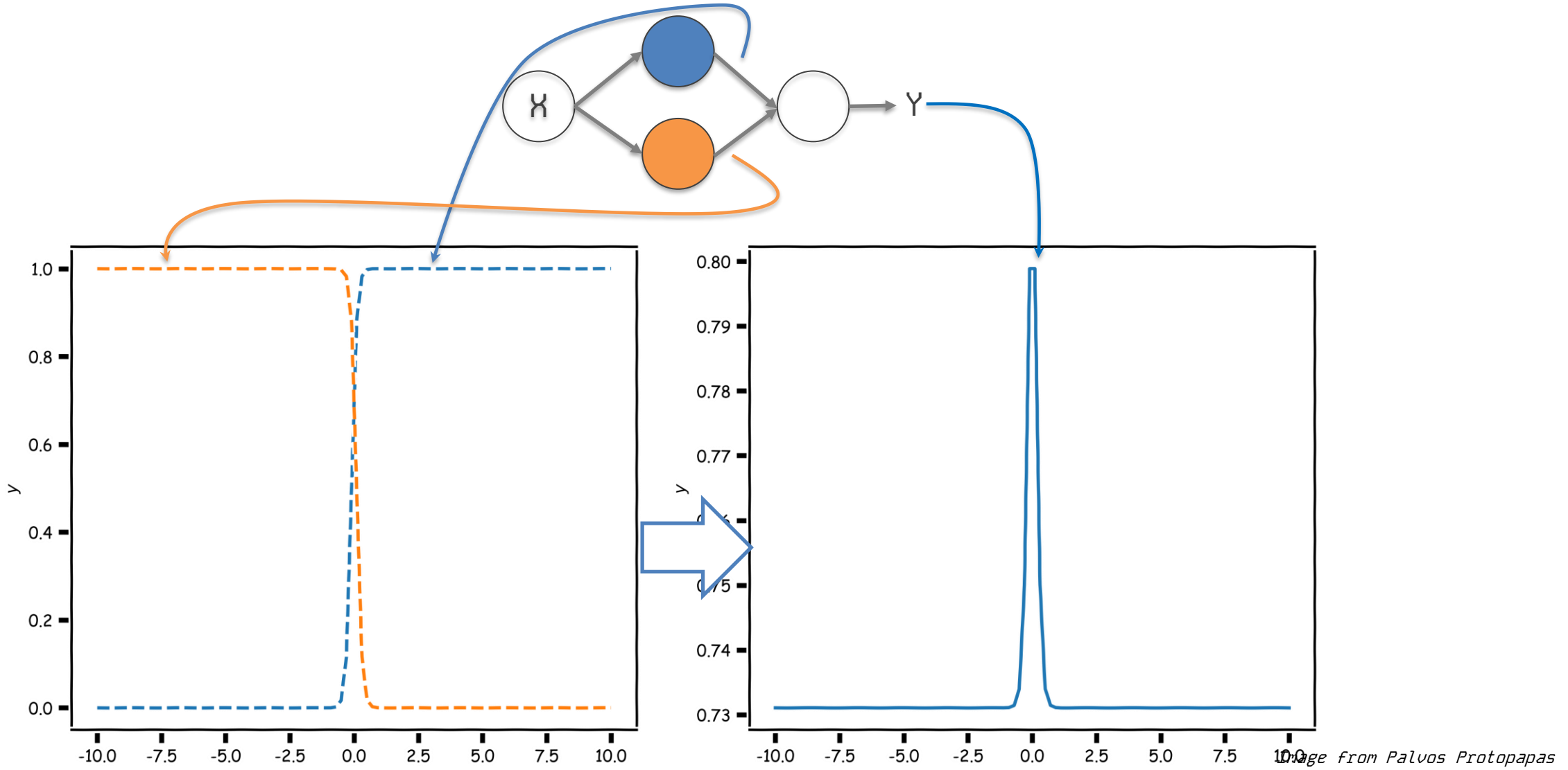


Image from Palvos Protopapas

Neural networks can model *any* reasonable function

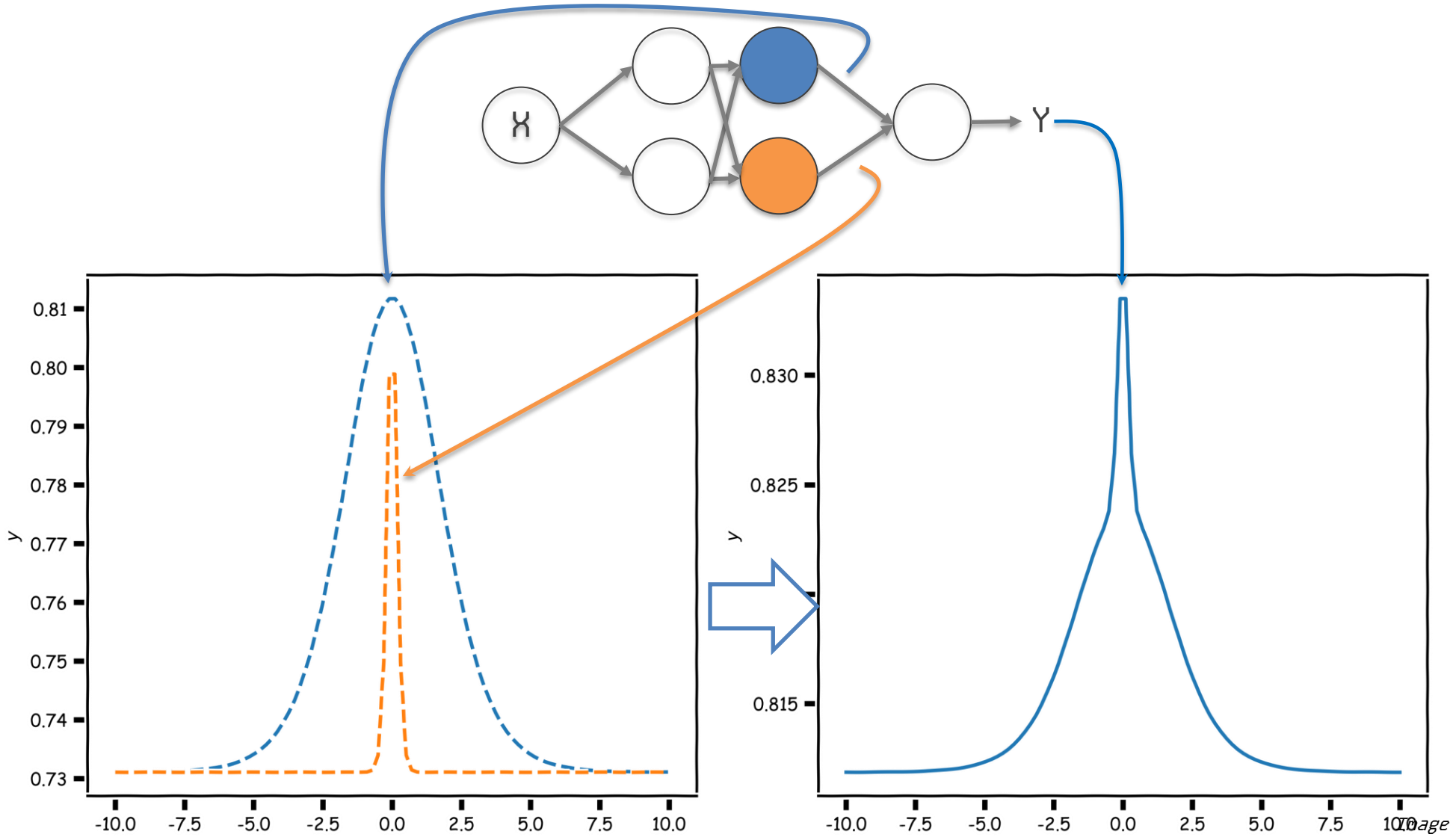


Image from Palvos Protopapas

SUPERVISE 6 SUPERVISE 7 SIMULATIONS

Adding layers allows us to model increasingly complex functions

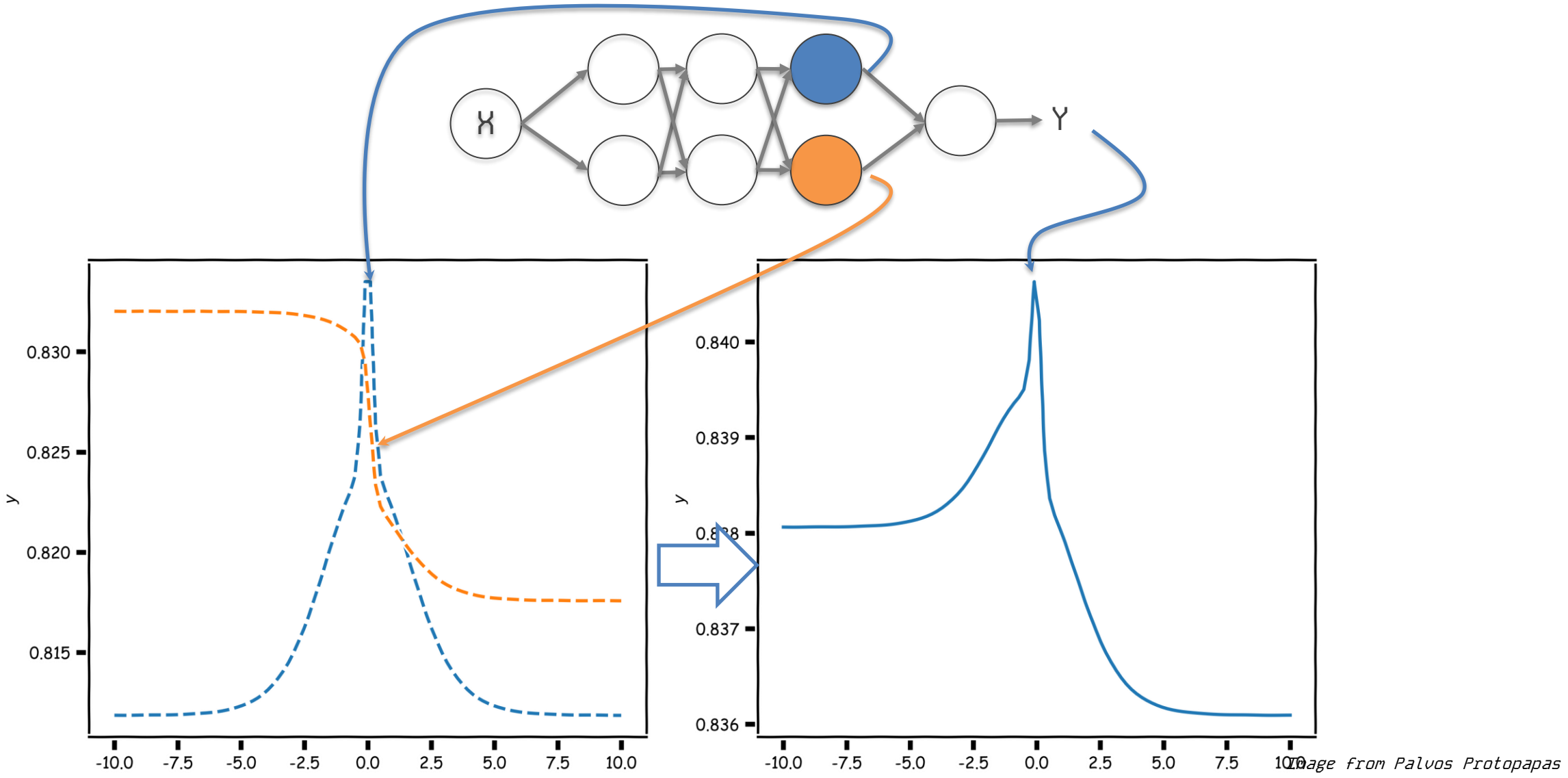


Image from Palvos Protopapas

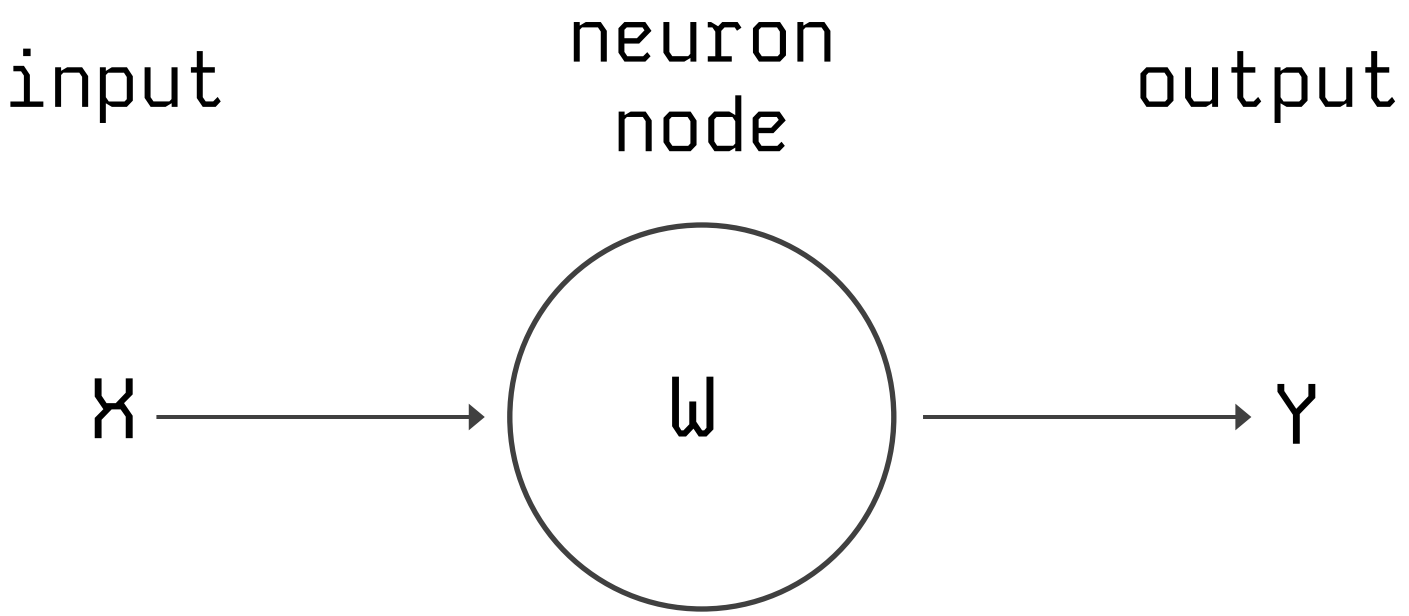
So far:

- A single neuron can be a logistic regression unit. We will soon see other choices.
- A neural network is a combination of logistic regression (or other types) units.
- A neural network can approximate non-linear functions.

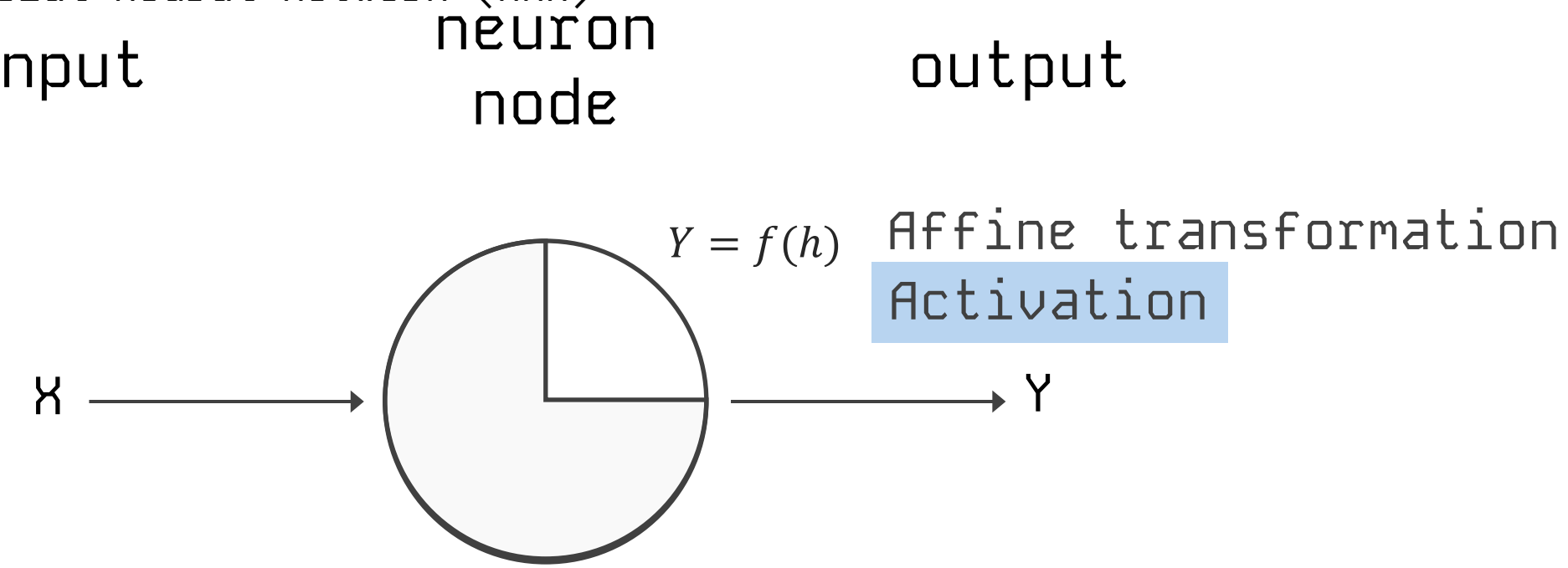
Next:

- What kind of activations, how many neurons, how many layers, output unit, and loss function?

Anatomy of artificial neural network (ANN)

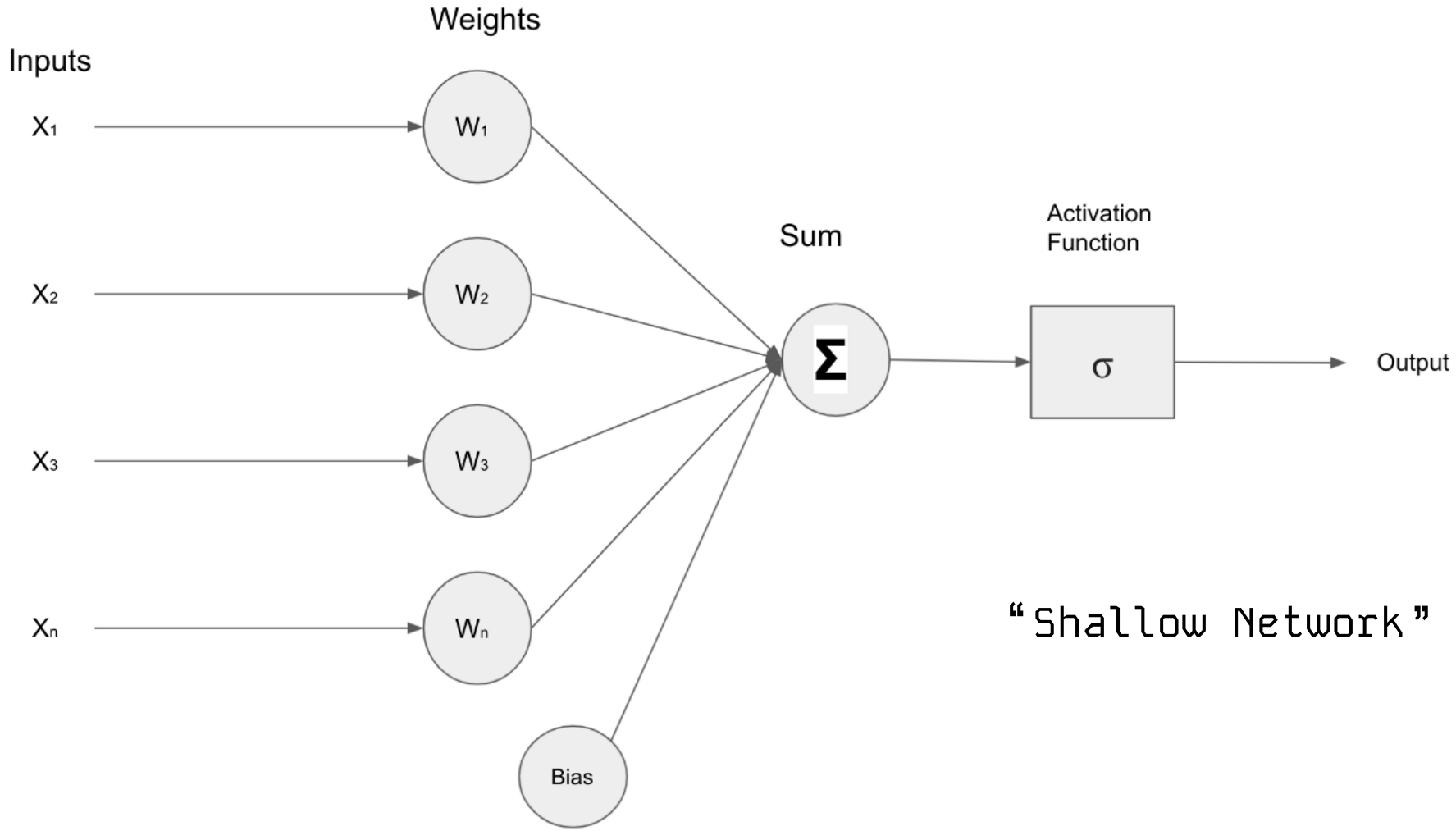


Anatomy of artificial neural network (ANN)

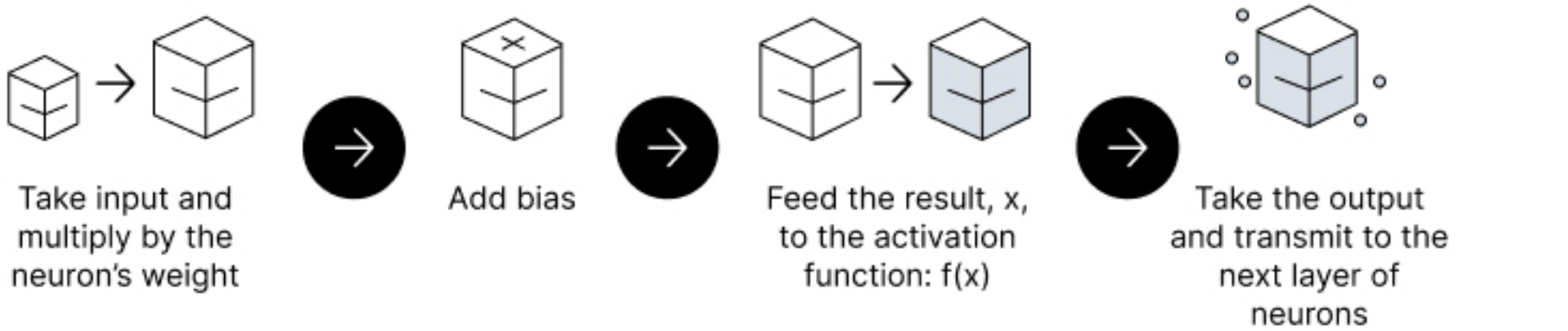


We will talk later about the choice of activation function. So far we have only talked about sigmoid as an activation function but there are other choices.

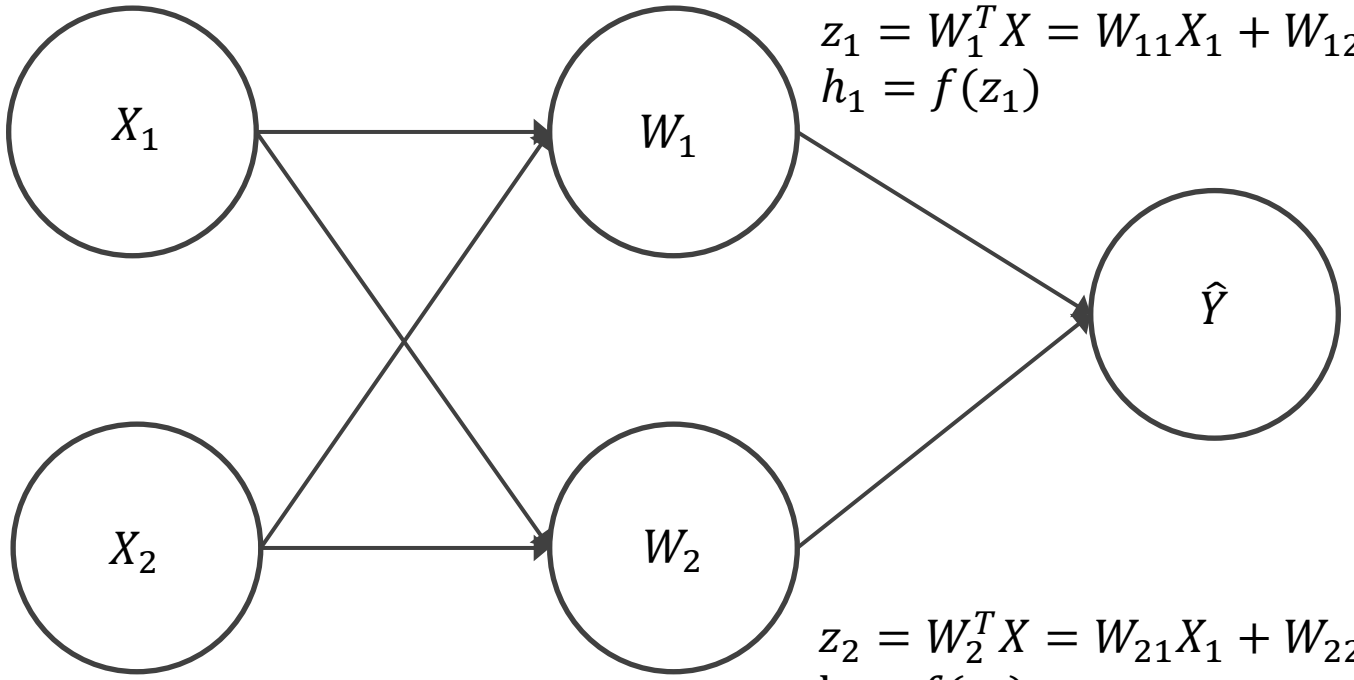
Neural Networks



"Shallow Network"


Take input and multiply by the neuron's weight
Add bias
Feed the result, x, to the activation function: f(x)
Take the output and transmit to the next layer of neurons
V7 Labs

Input layer hidden layer output layer



$z_1 = W_1^T X = W_{11}X_1 + W_{12}X_2 + W_{10}$

$h_1 = f(z_1)$

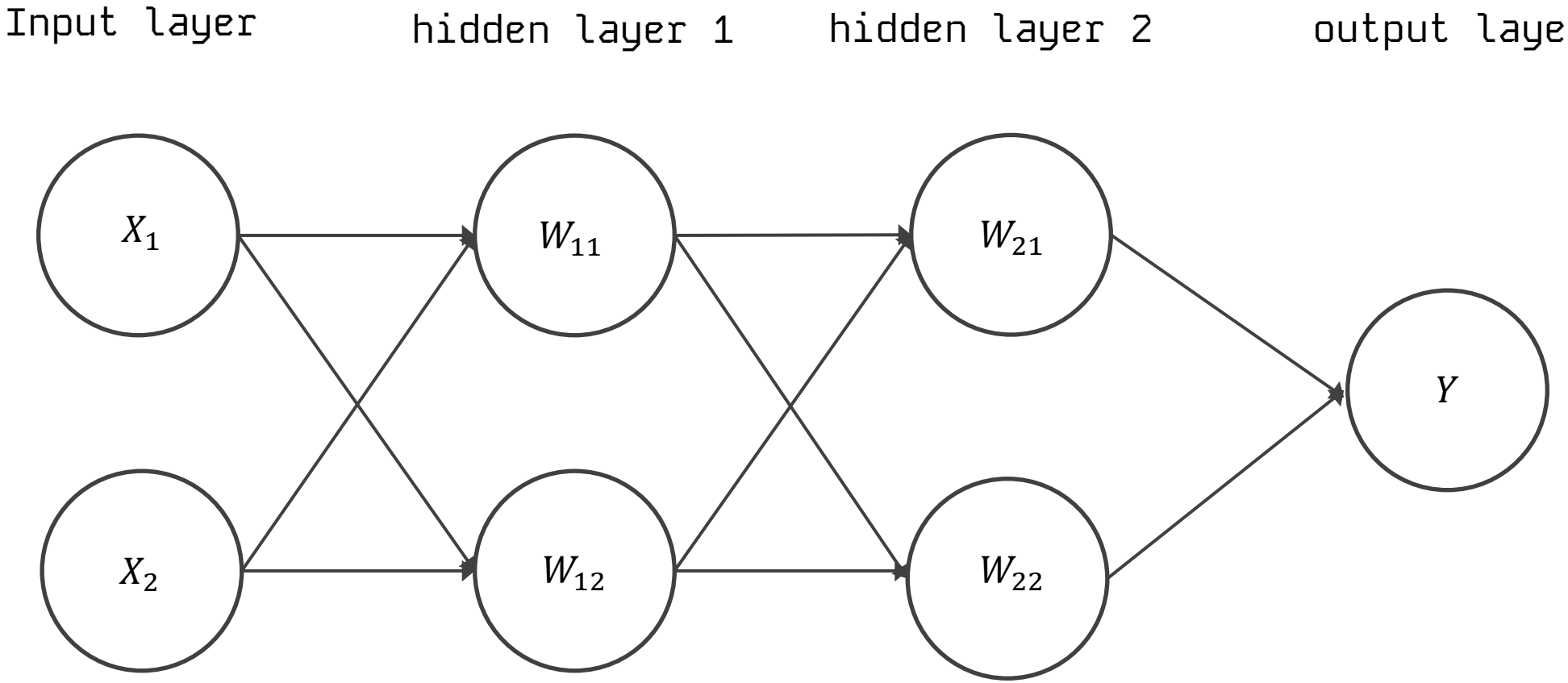
$z_2 = W_2^T X = W_{21}X_1 + W_{22}X_2 + W_{20}$

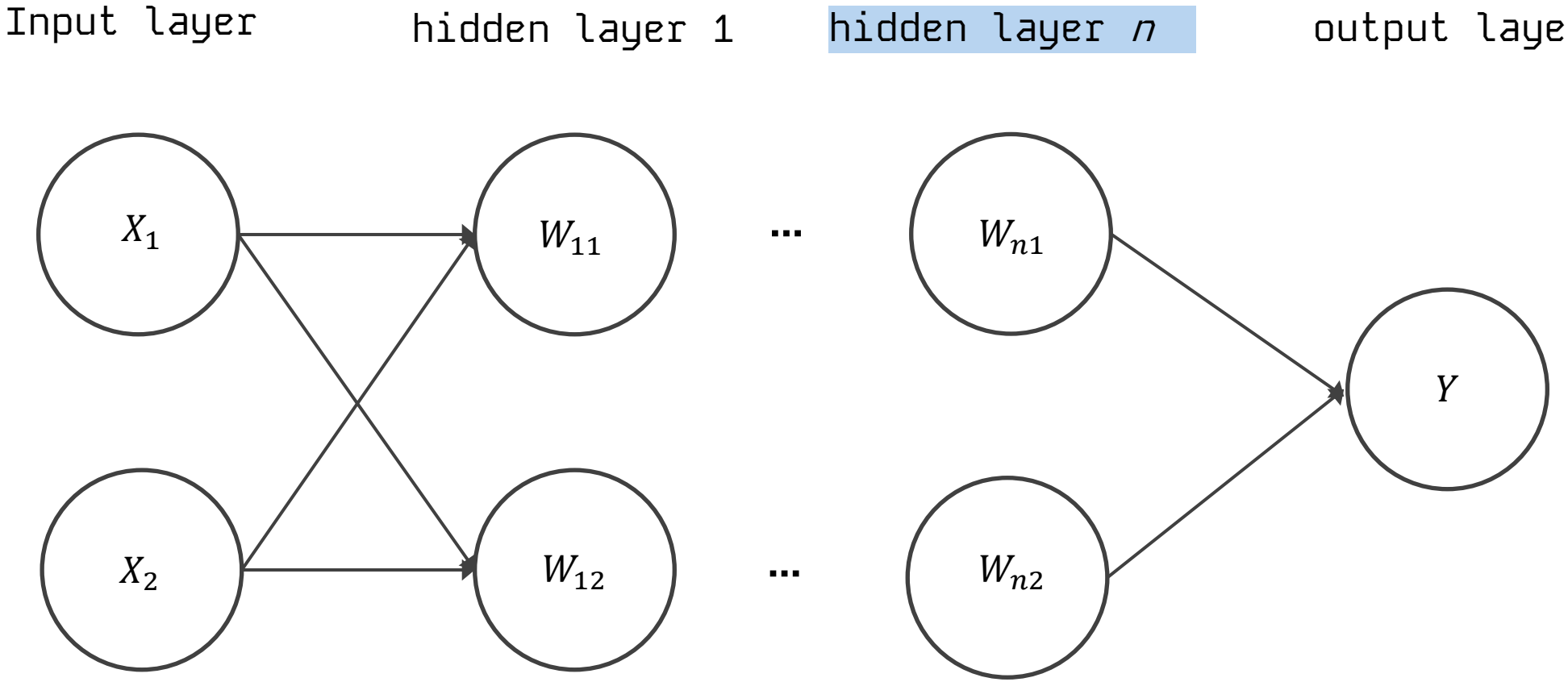
$h_2 = f(z_2)$

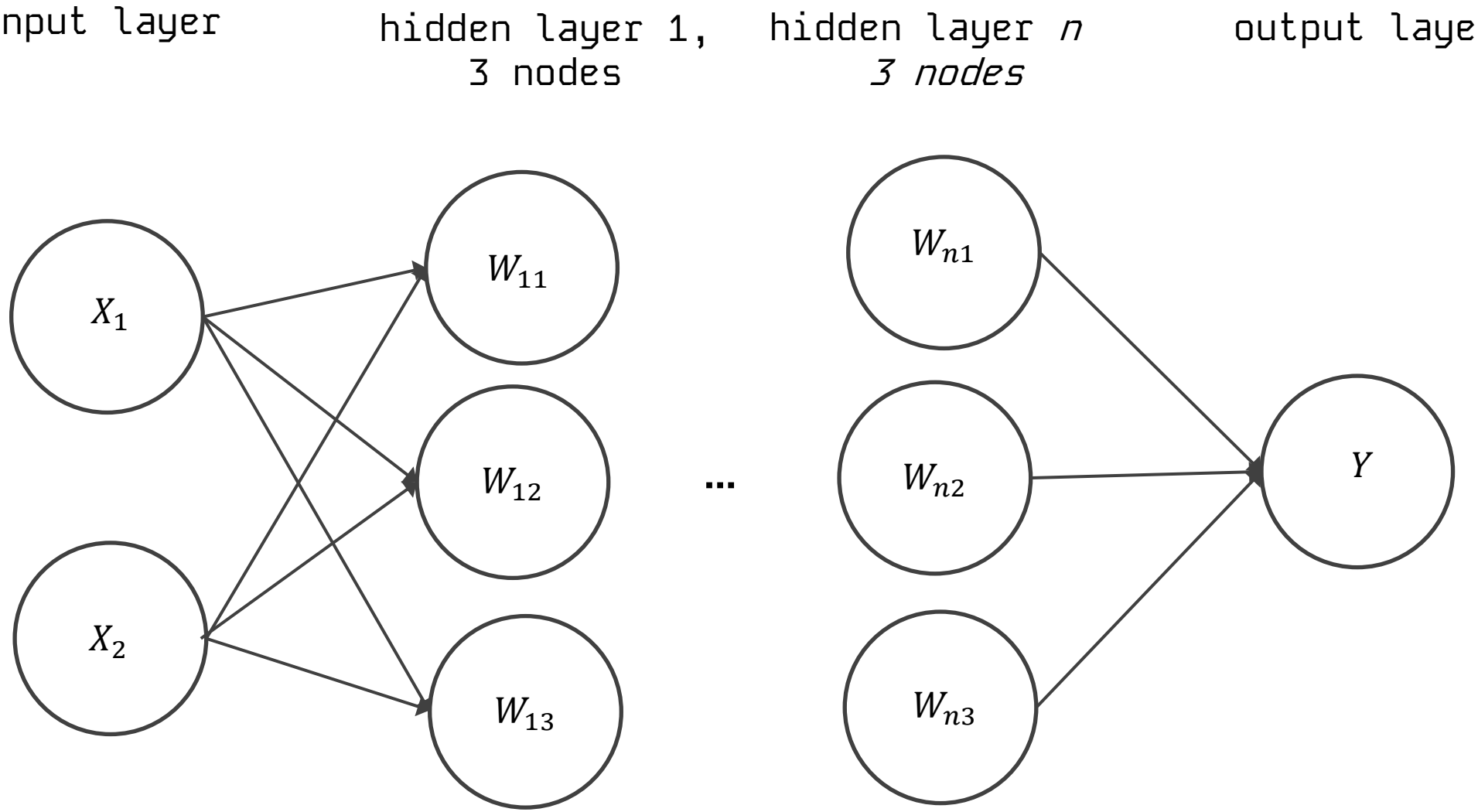
$\hat{Y} = g(h_1, h_2) \longrightarrow J = \mathcal{L}(\hat{Y}, Y)$

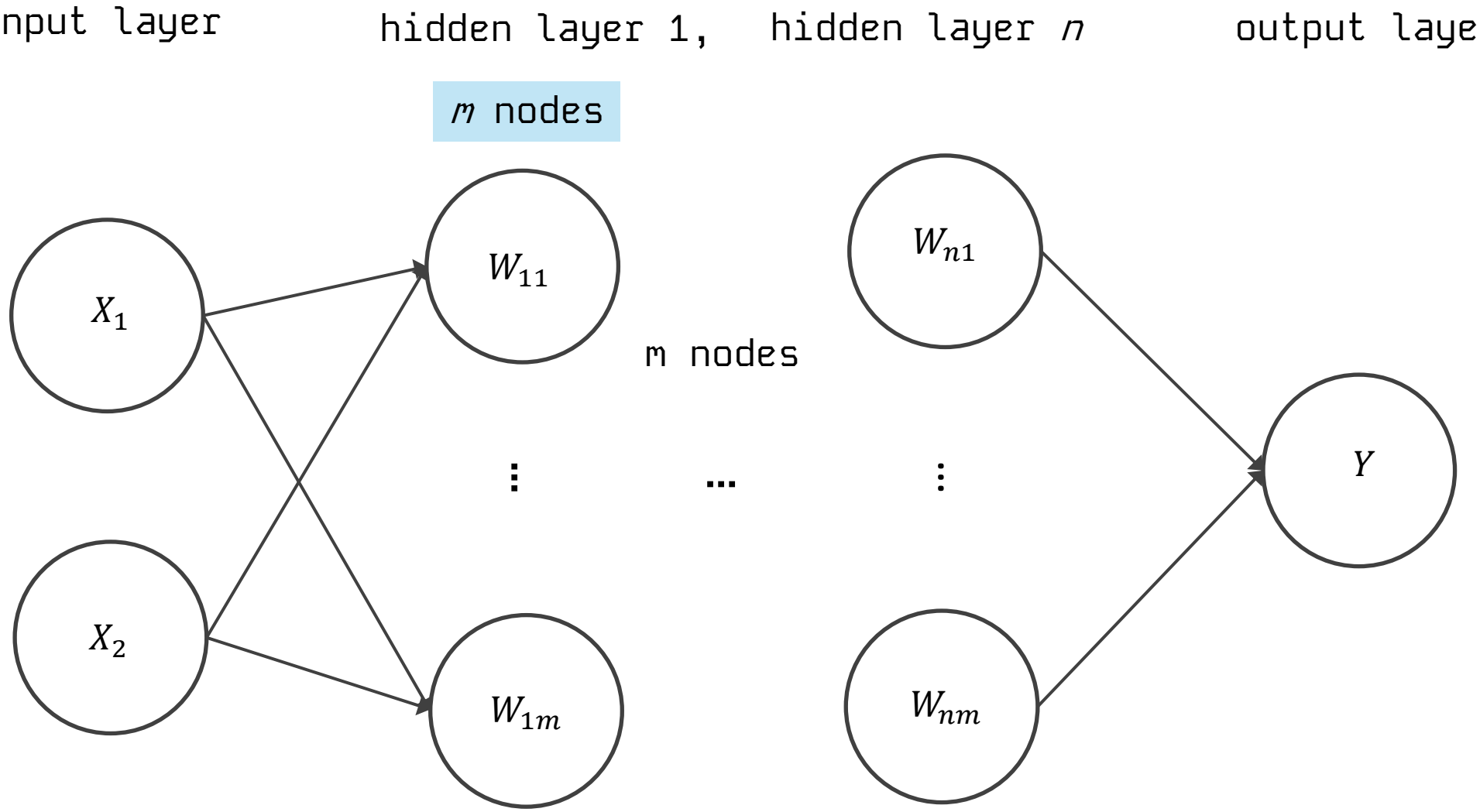
Output function

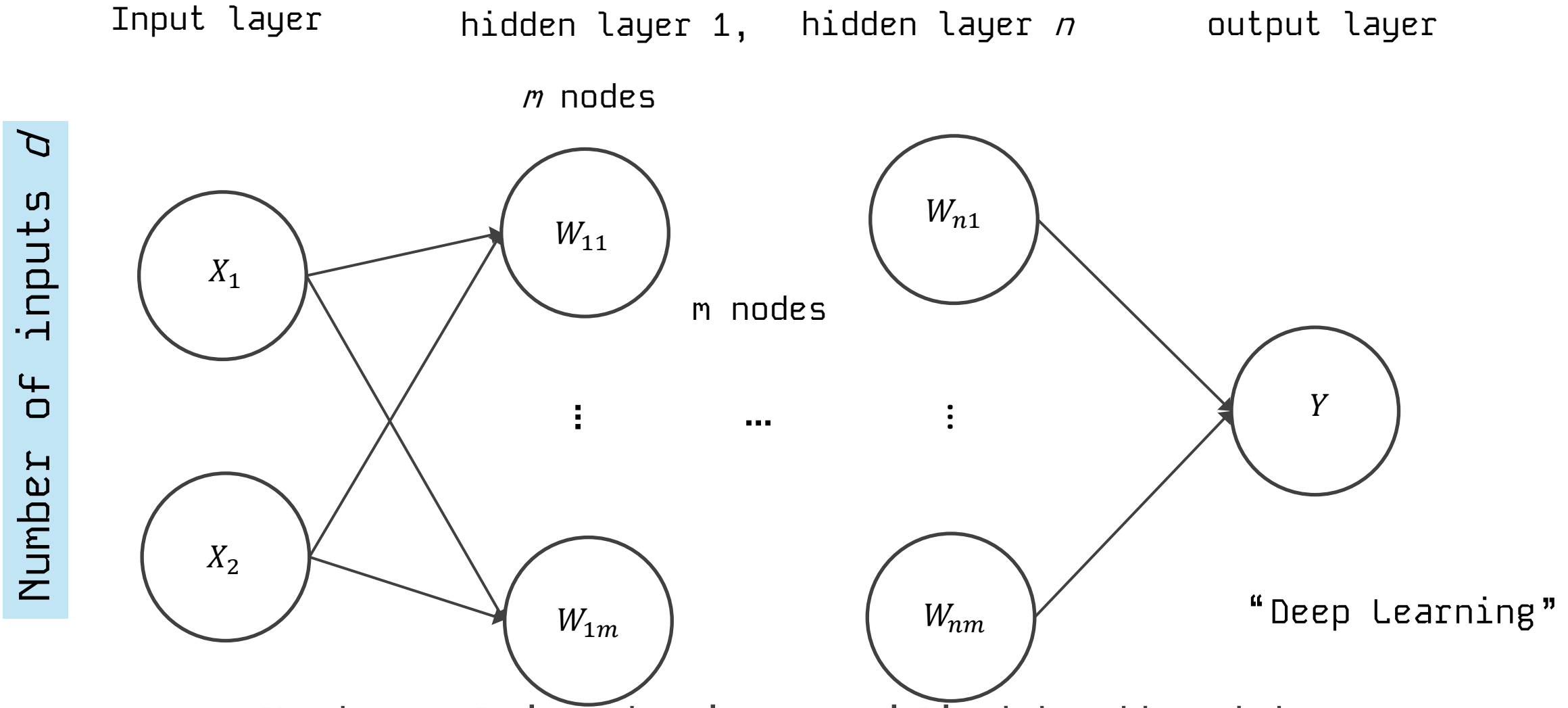
Loss function


Input layer
hidden layer 1
hidden layer 2
output layer
X_1
X_2
W_{11}
W_{12}
W_{21}
W_{22}
Y


Input layer
hidden layer 1
hidden layer n
output layer
X_1
X_2
W_{11}
W_{12}
W_{n1}
W_{n2}
Y

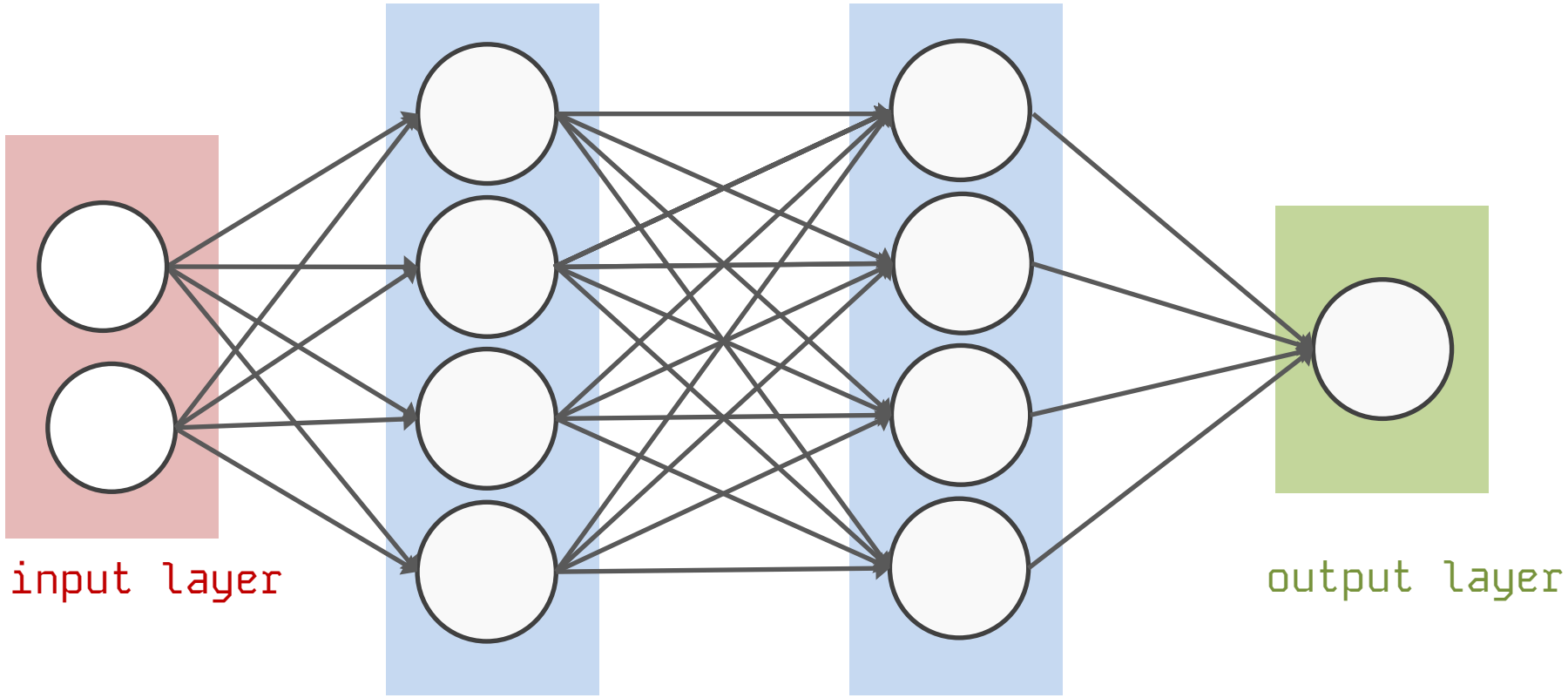

Input layer
hidden layer 1,
3 nodes
hidden layer *n*
3 nodes
output layer
X_1
X_2
W_{11}
W_{12}
W_{13}
...
W_{n1}
W_{n2}
W_{n3}
Y


Input layer
hidden layer 1,
hidden layer n
output layer
m nodes
X_1
X_2
W_{11}
W_{1m}
m nodes
W_{n1}
W_{nm}
Y



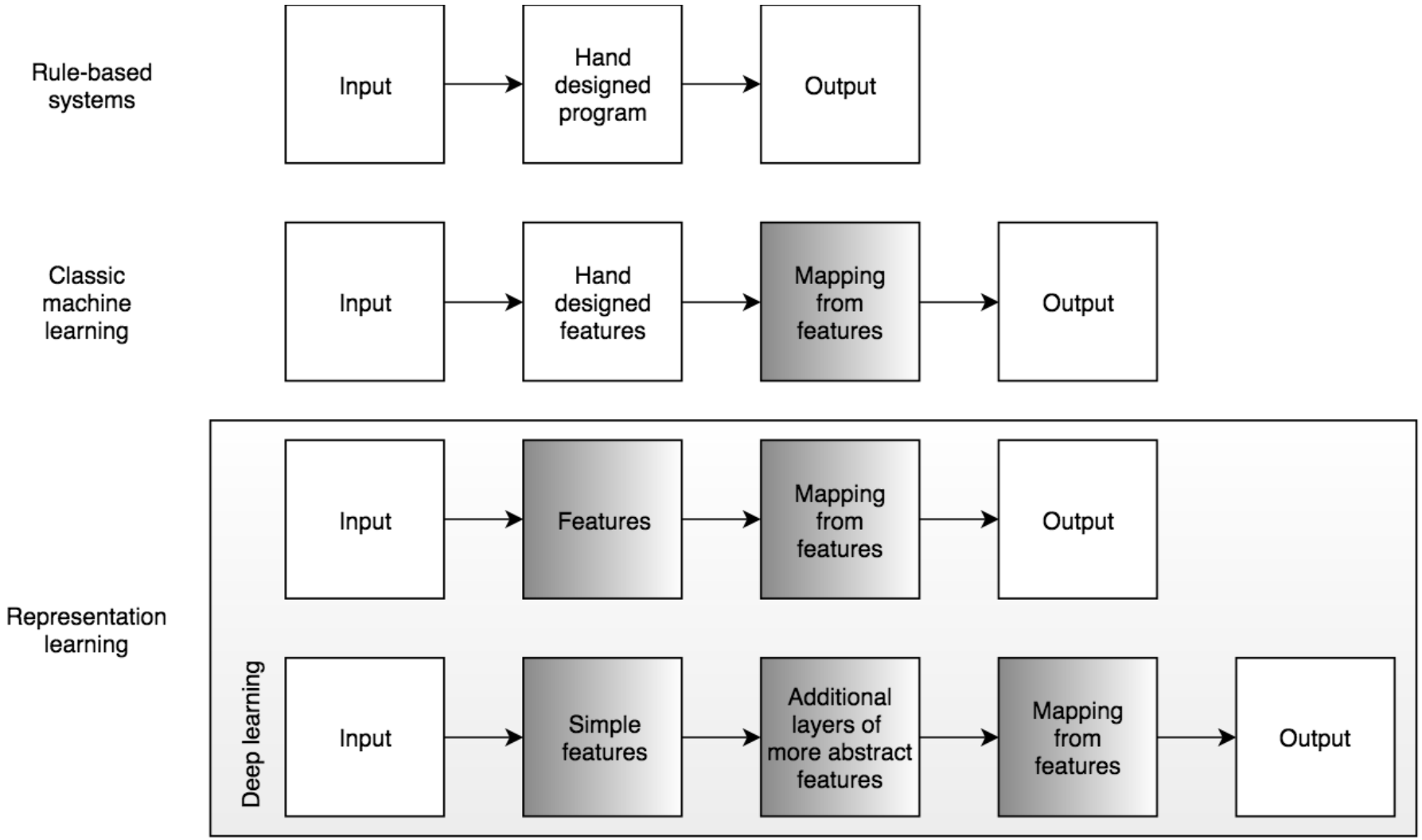
Number of inputs is specified by the data

Neural Networks

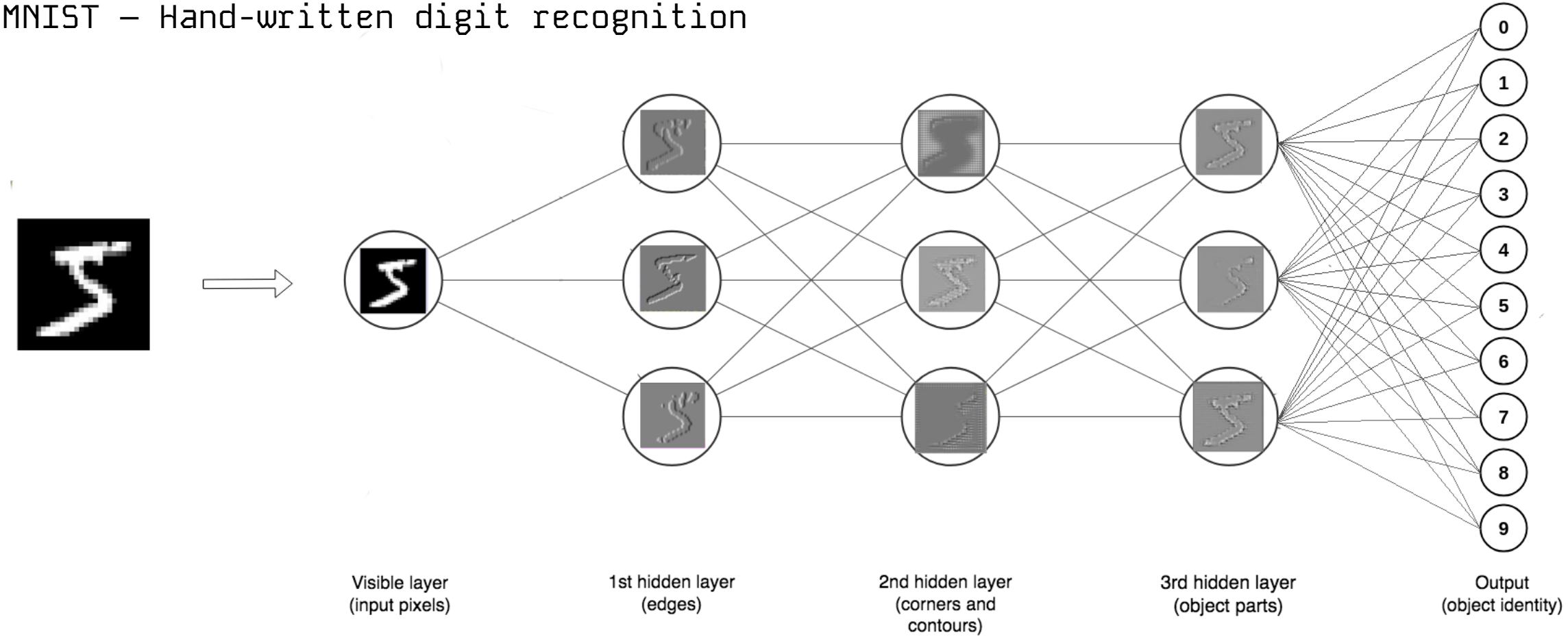


https://bit.ly/BPS5231-GoogleNN

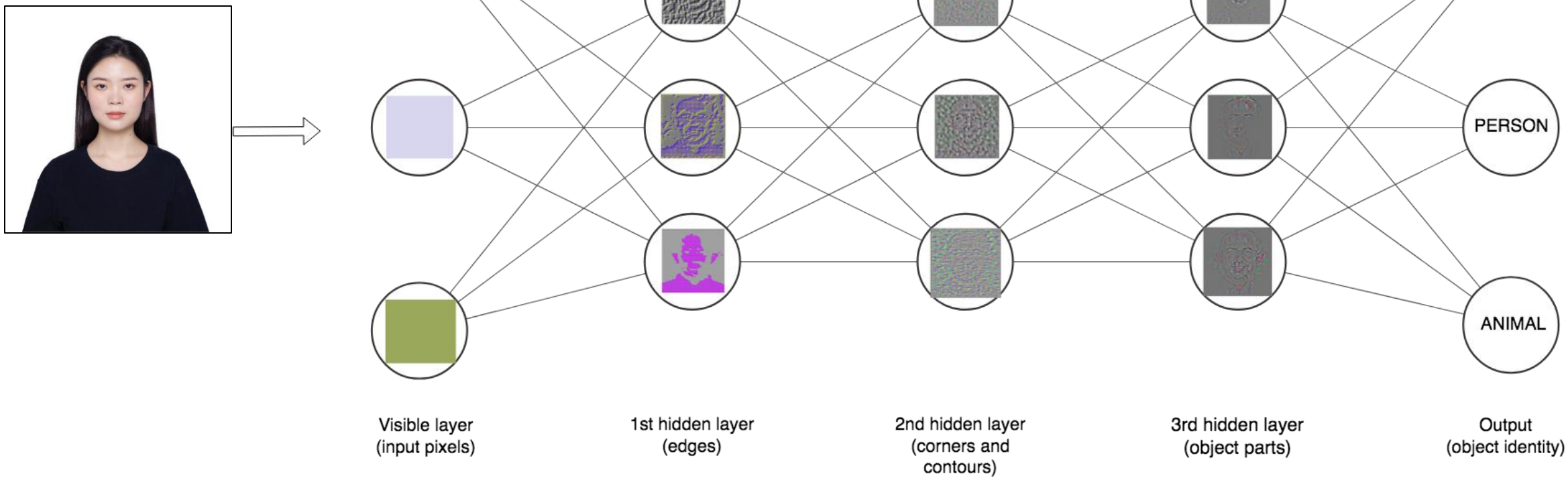
Neural Networks



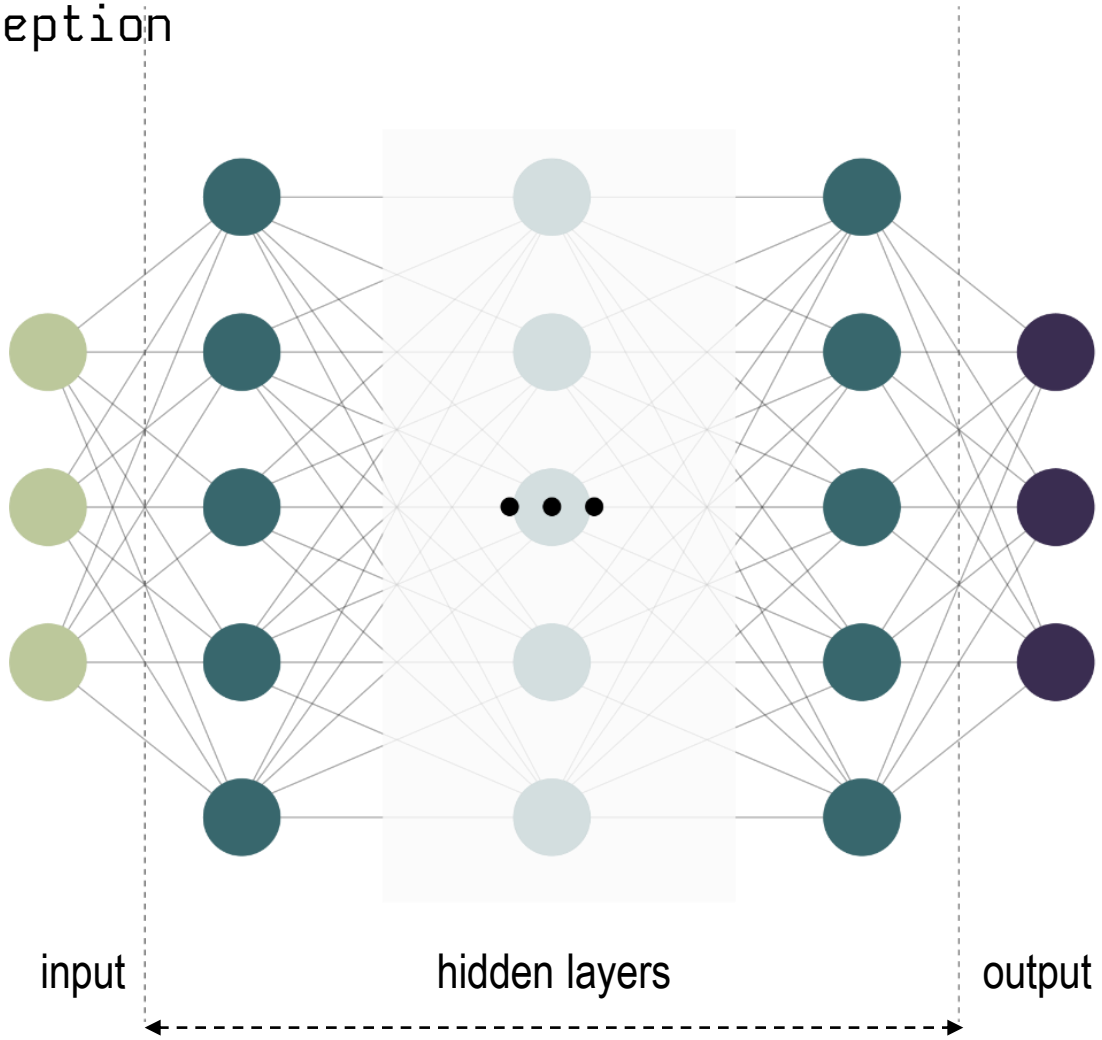
MNIST – Hand-written digit recognition



Neural Networks



MLP – Multi-Layer Perception



Google Colab
https://bit.ly/BPS5231-L7

Neural Networks

$S_n = \{(\boldsymbol{x}^{(t)}, \boldsymbol{y}^{(t)}), t = 1, \ldots, n\}$ = dataset with features and labels (ground truth examples)

$\mathcal{H}$ = hypothesis class ⇒ a neural network architecture is a space of functions

$\mathcal{L}$ = loss function (often squared error for regression) e.g. $\mathcal{L}(x_1, x_2) = (x_1 - x_2)^2$

Goal: pick a function $h \in \mathcal{H}$ that predicts labels y based on features x as accurately as possible
⇒ Optimize the weights (always) and the architecture (sometimes) of a neural network

observation

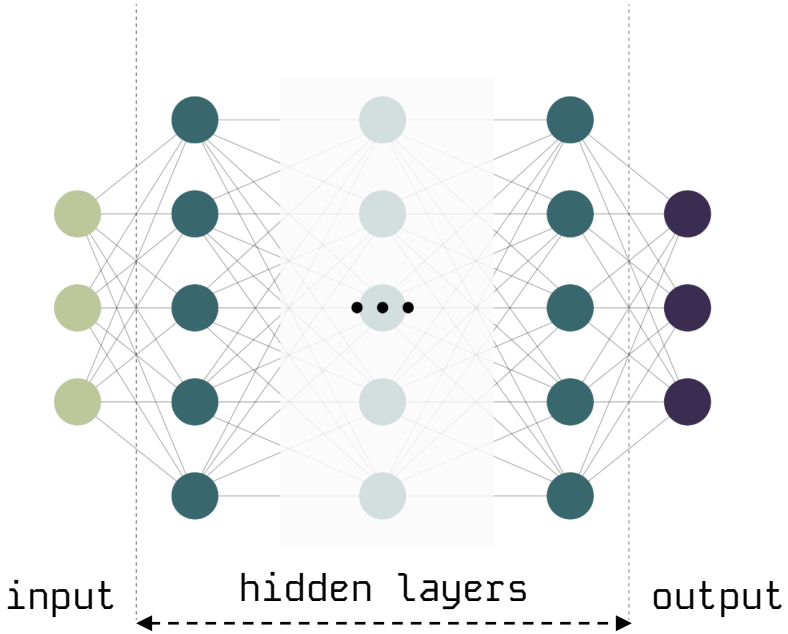
$$\min_{h \in \mathcal{H}} \sum_{i=1}^{n} \mathcal{L}(h(x_i), y_i)$$

prediction

loss NN weights

$$\min_{W} \sum_{i=1}^{n} \mathcal{L}(F(x_i, W), y_i)$$

prediction observation



$$h = f(W^T X + b)$$

The activation function should:

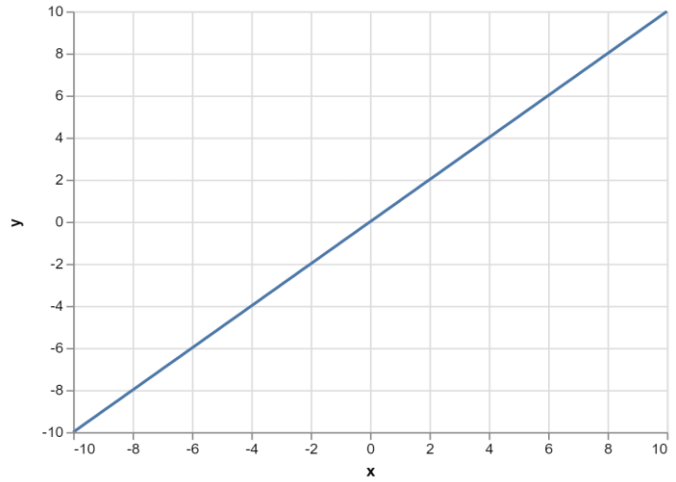
- Provide non-linearity
- Ensure gradients remain large through hidden unit

Common choices are

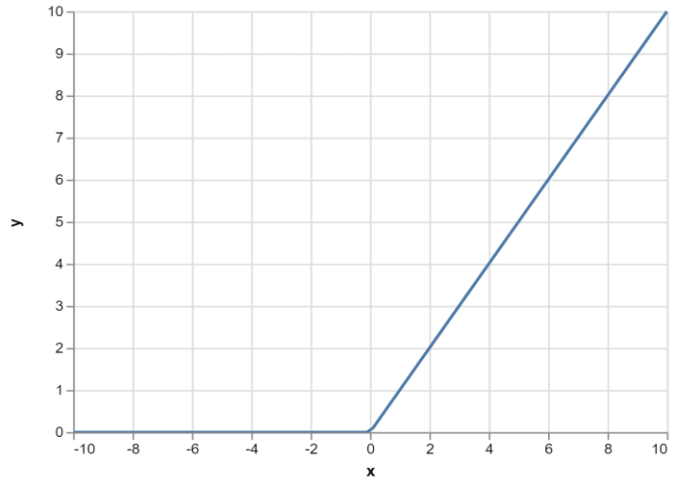
- Sigmoid
- Relu, leaky ReLU, Generalized ReLU, MaxOut
- softplus
- tanh
- swish

Neural Networks (Activation Function)

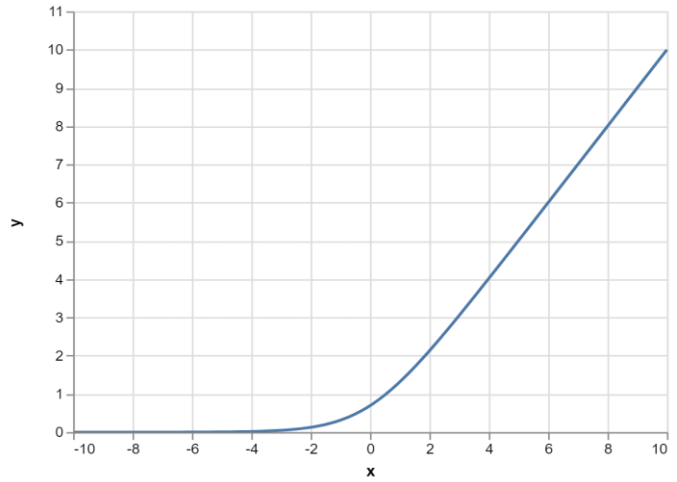
Linear



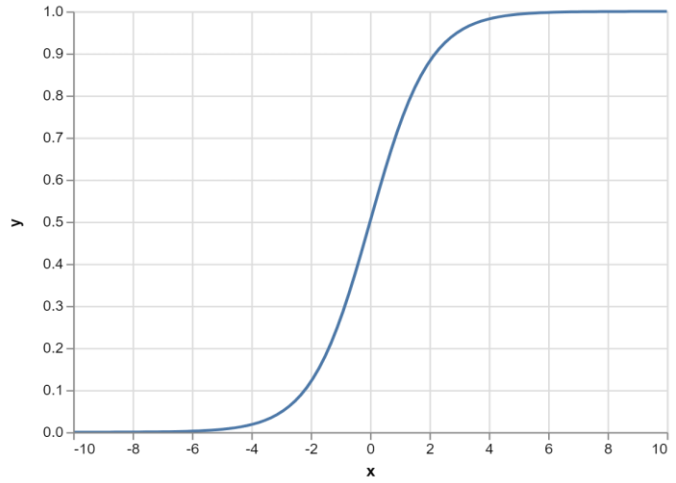
ReLU



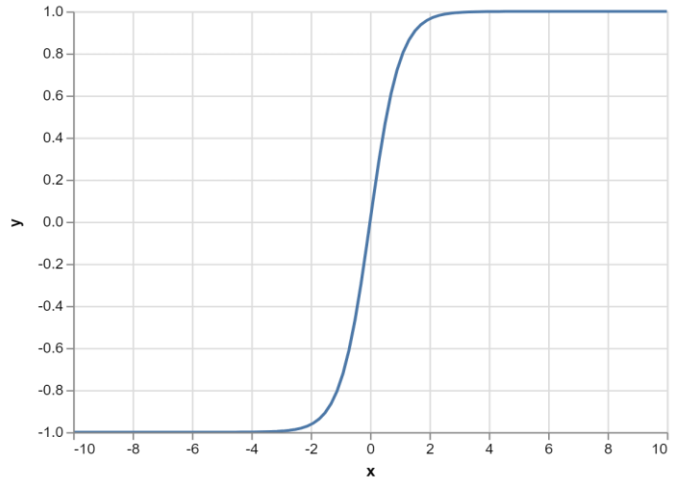
Softplus



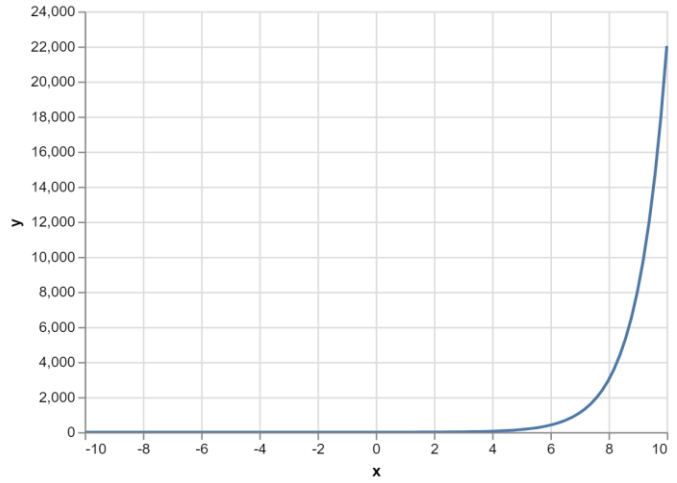
Sigmoid



Tanh



Exp

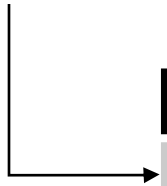


Neural Networks (Output Units & Loss Functions)

Output Type	Output Distribution	Output layer	Loss Function
Binary			

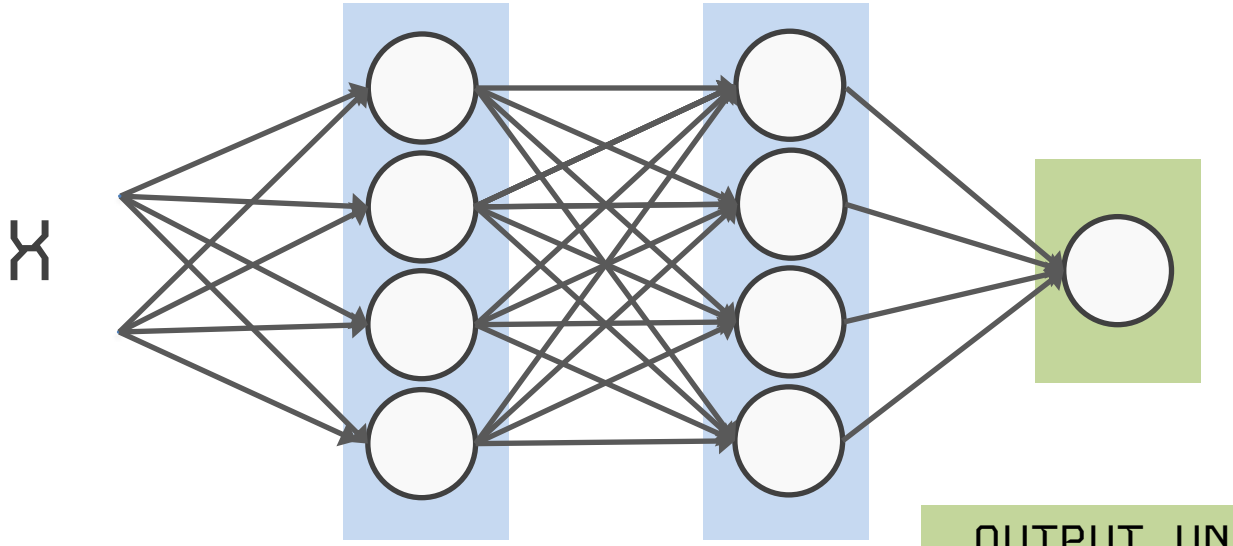
Neural Networks (Output Units & Loss Functions)

Examples
- Overheating risk next day (zone will exceed 32 °C for ≥2 h: Yes/No)
- Economizer fault detected from sensors (stuck damper: Yes/No)
- Meets daylight code (sDA300/50% ≥ target: Yes/No)
- Retrofit passes payback (≤5-year simple payback: Yes/No)



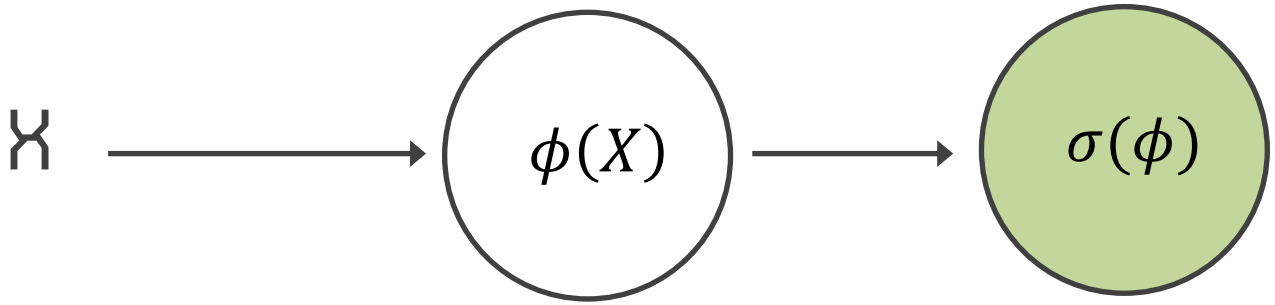
Output Type	Output Distribution	Output layer	Loss Function
Binary	Bernoulli		Binary Cross Entropy

Neural Networks (Output Units & Loss Functions)



$$\hat{Y} = \mathrm{P}(\mathrm{y} = 1)$$

OUTPUT UNIT for binary classification



$$\hat{Y} = \mathrm{P}(\mathrm{y} = 1)$$

$$X \Longrightarrow \phi(X) \Longrightarrow P(y = 1) = \frac{1}{1 + e^{-\phi(X)}}$$

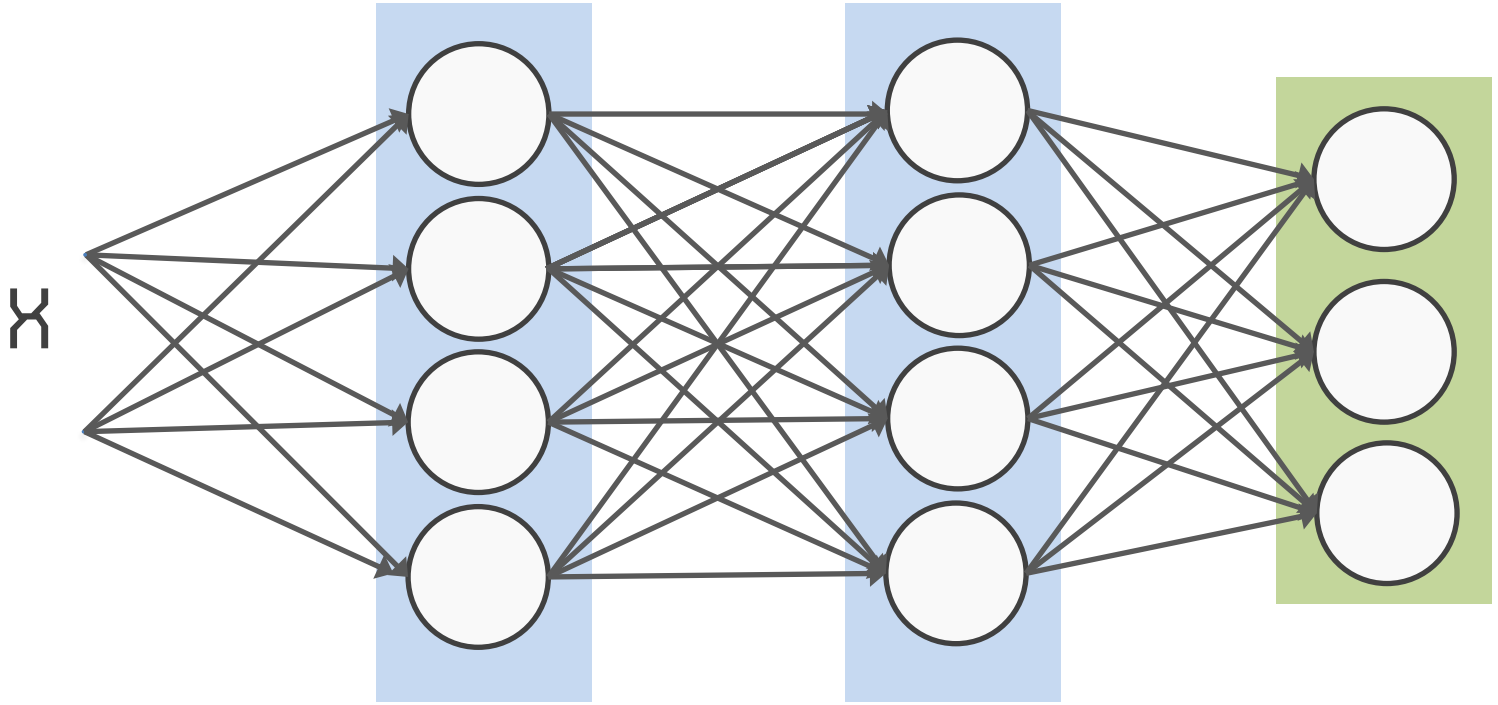
Neural Networks (Output Units & Loss Functions)

Output Type	Output Distribution	Output layer	Loss Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy

Neural Networks (Output Units & Loss Functions)

Output Type	Output Distribution	Output layer	Loss Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy
Discrete	Multinouli	?	Cross Entropy

Neural Networks (Output Units & Loss Functions)



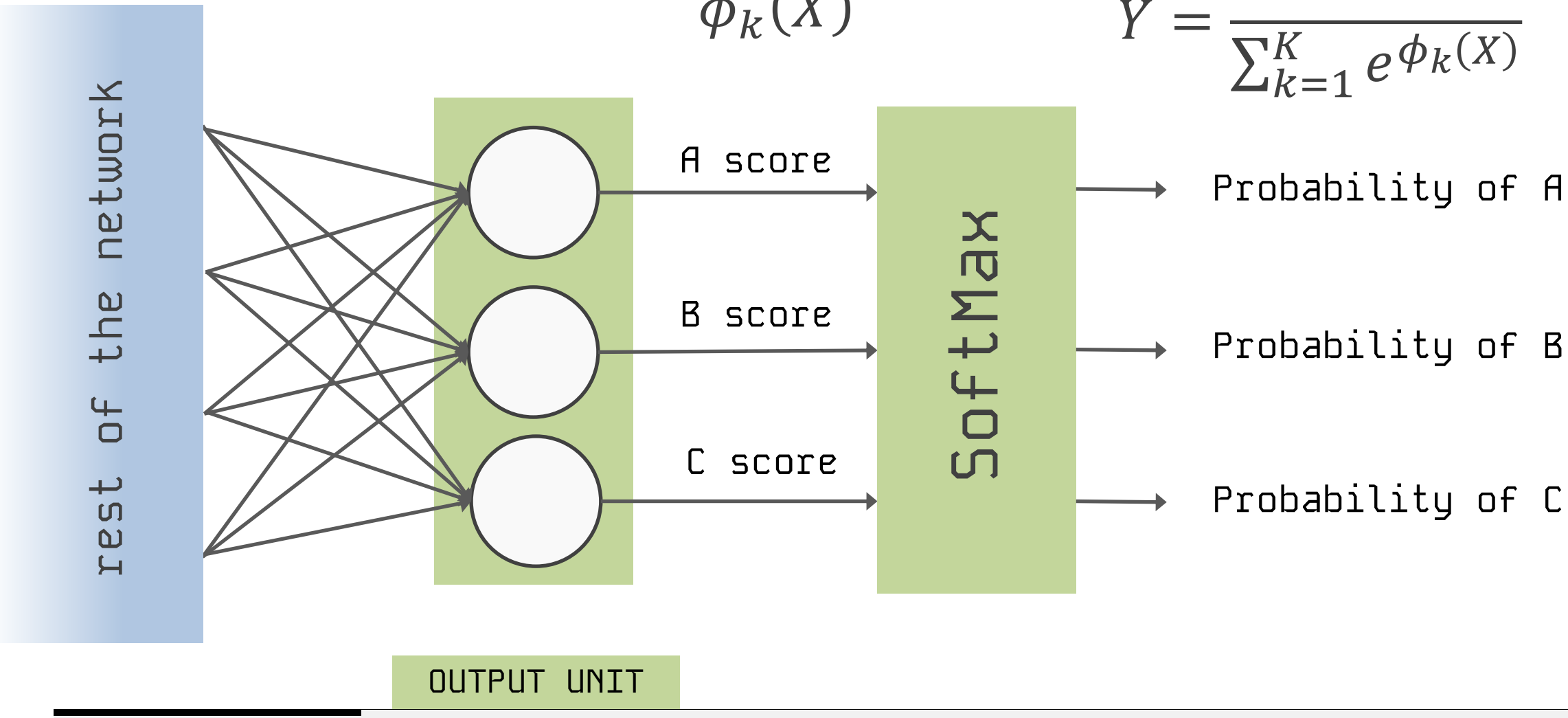
$$\hat{Y} = [P_1, P_2, P_3]$$

OUTPUT UNIT

SoftMax

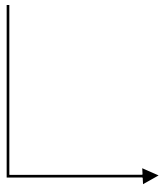
$$\phi_k(X)$$

$$\hat{Y} = \frac{e^{\phi_k(X)}}{\sum_{k=1}^{K} e^{\phi_k(X)}}$$



SoftMax

- Façade material class from images (glass / concrete / brick / mixed) for embodied-carbon lookup
- HVAC operating mode (cooling / heating / economizer / vent-only)
- Urban typology (tower / slab / courtyard / row) for massing & wind assumptions
- Thermal comfort category (A / B / C) for compliance reporting

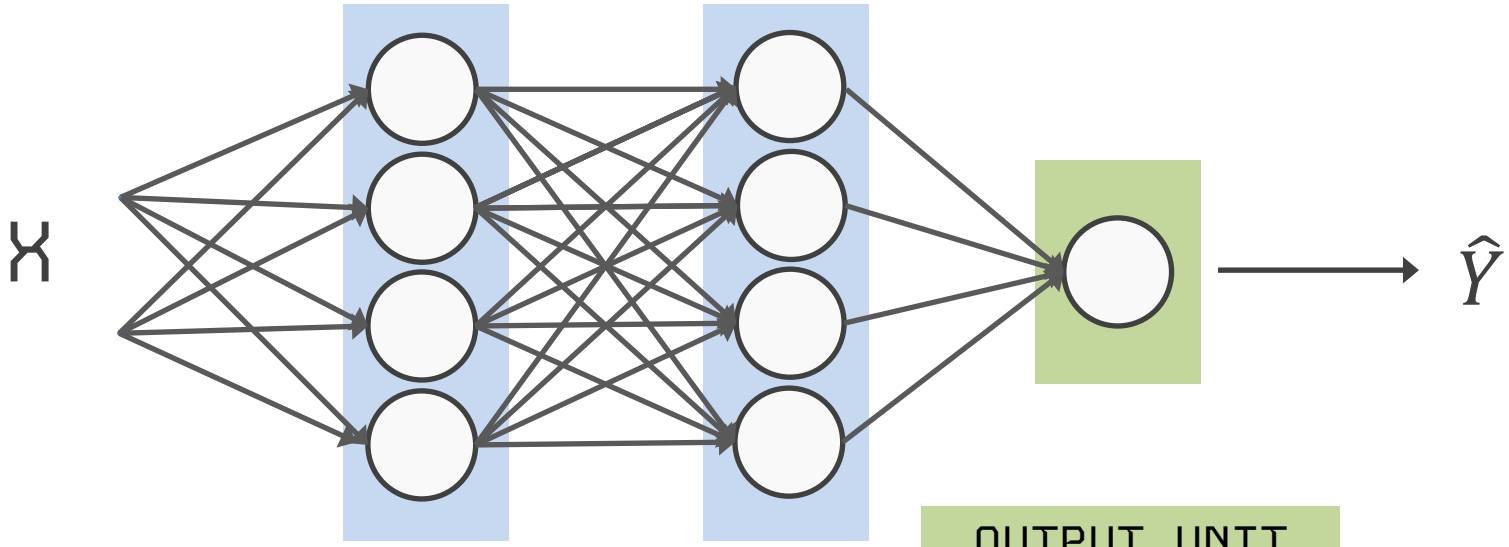


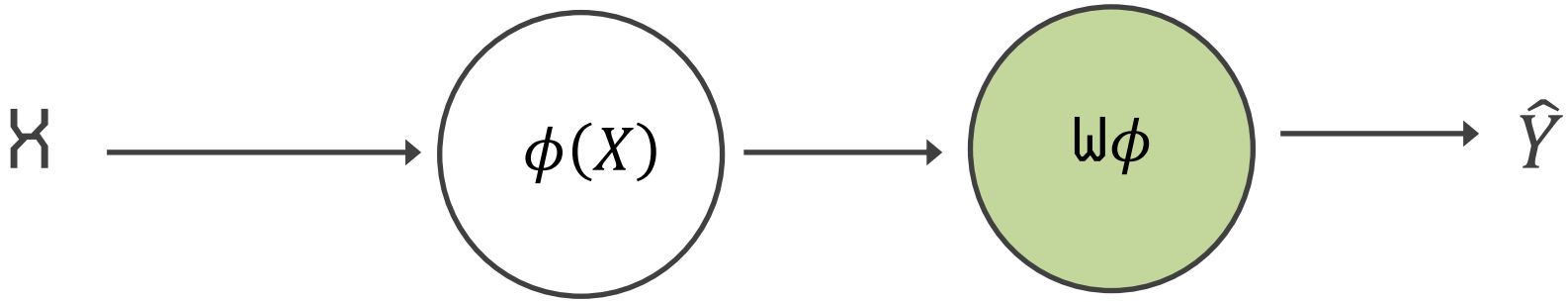
Output Type	Output Distribution	Output layer	Loss Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy
Discrete	Multinouli	Softmax	Cross Entropy

Neural Networks (Output Units & Loss Functions)

Output Type	Output Distribution	Output layer	Loss Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy
Discrete	Multinouli	Softmax	Cross Entropy
Continuous	Gaussian	?	MSE

Neural Networks (Output Units & Loss Functions)





$$X \Longrightarrow \phi(X) \Longrightarrow \hat{Y} = W\phi(X)$$

Linear

- Hourly cooling electric demand (kW)
- Annual EUI ($kWh \cdot m^{-2} \cdot yr^{-1}$) or carbon intensity ($kgCO_2e \cdot m^{-2} \cdot yr^{-1}$)
- Next-hour operative temperature (°C) in a naturally ventilated room
- CO_2 concentration forecast (ppm) for IAQ control

Output Type	Output Distribution	Output layer	Loss Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy
Discrete	Multinouli	Softmax	Cross Entropy
Continuous	Gaussian	Linear	MSE

Module: tf.keras.losses

DO NOT EDIT.

This file was autogenerated. Do not edit it by hand, since your modifications would be overwritten.

Classes

`class BinaryCrossentropy`: Computes the cross-entropy loss between true labels and predicted labels.

`class BinaryFocalCrossentropy`: Computes focal cross-entropy loss between true labels and predictions.

`class CTC`: CTC (Connectionist Temporal Classification) loss.

`class CategoricalCrossentropy`: Computes the crossentropy loss between the labels and predictions.

`class CategoricalFocalCrossentropy`: Computes the alpha balanced focal crossentropy loss.

`class CategoricalHinge`: Computes the categorical hinge loss between `y_true` & `y_pred`.

`class CosineSimilarity`: Computes the cosine similarity between `y_true` & `y_pred`.

`class Dice`: Computes the Dice loss value between `y_true` and `y_pred`.

`class Hinge`: Computes the hinge loss between `y_true` & `y_pred`.

`class Huber`: Computes the Huber loss between `y_true` & `y_pred`.

`class KLDivergence`: Computes Kullback-Leibler divergence loss between `y_true` & `y_pred`.

`class LogCosh`: Computes the logarithm of the hyperbolic cosine of the prediction error.

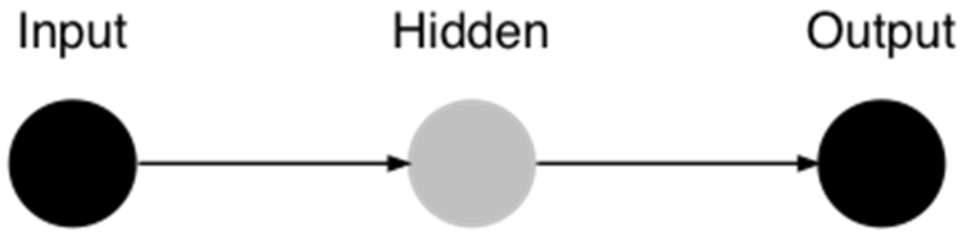
`class Loss`: Loss base class.

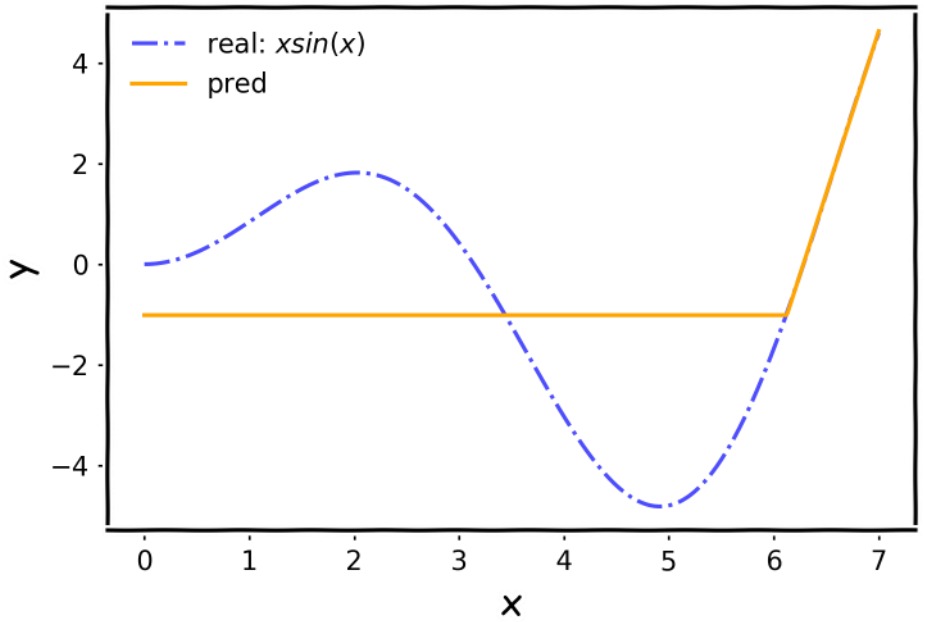
`class MeanAbsoluteError`: Computes the mean of absolute difference between labels and predictions.

`class MeanAbsolutePercentageError`: Computes the mean absolute percentage error between `y_true` & `y_pred`.

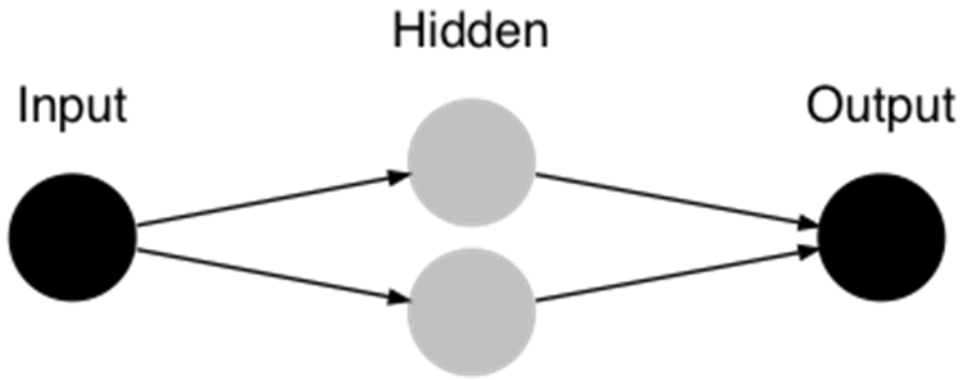
`class MeanSquaredError`: Computes the mean of squares of errors between labels and predictions.

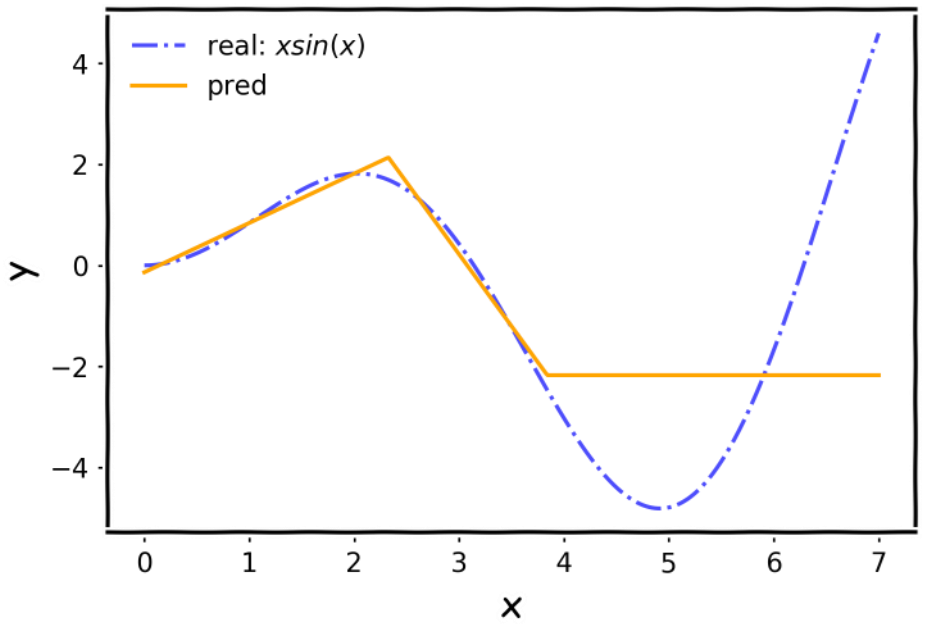
Neural Networks (Architecture / Non-linearity)



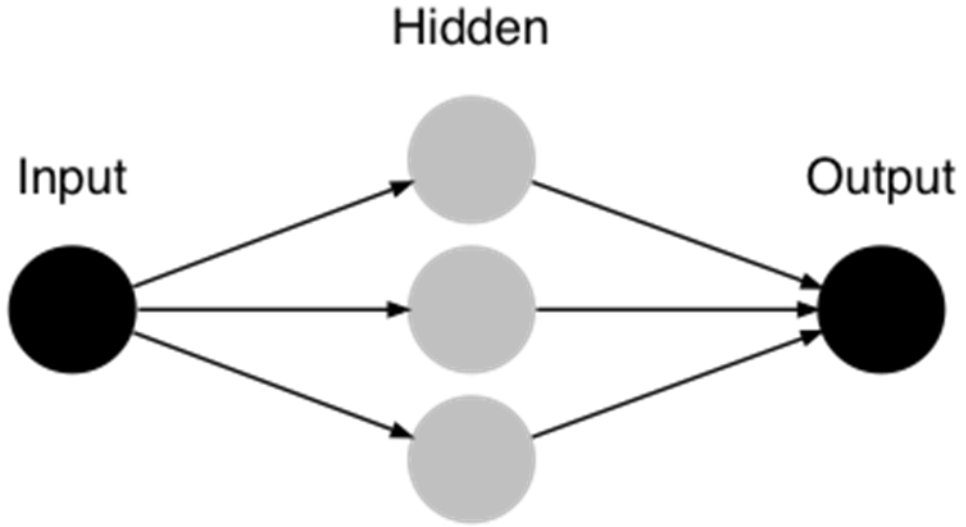


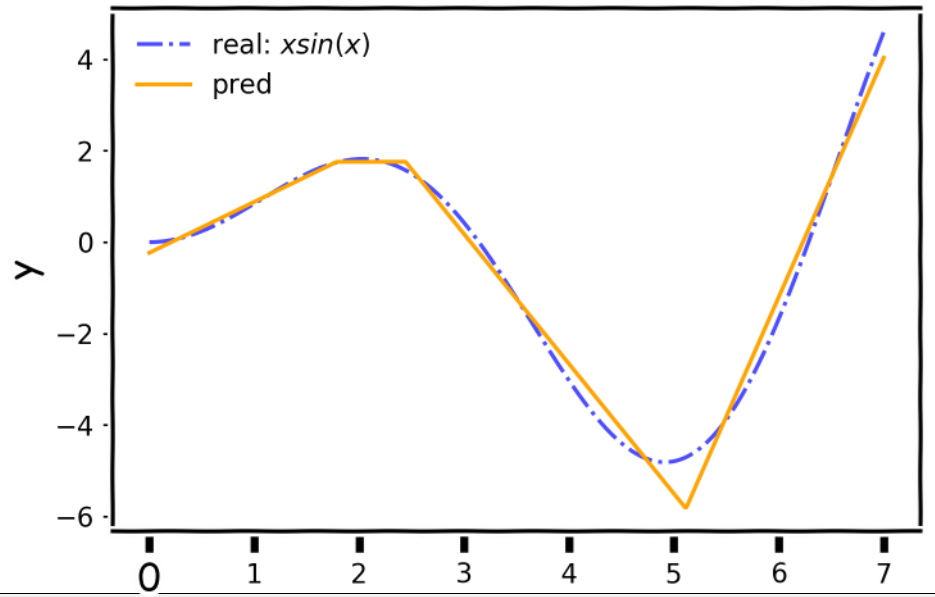
Neural Networks (Architecture / Non-linearity)



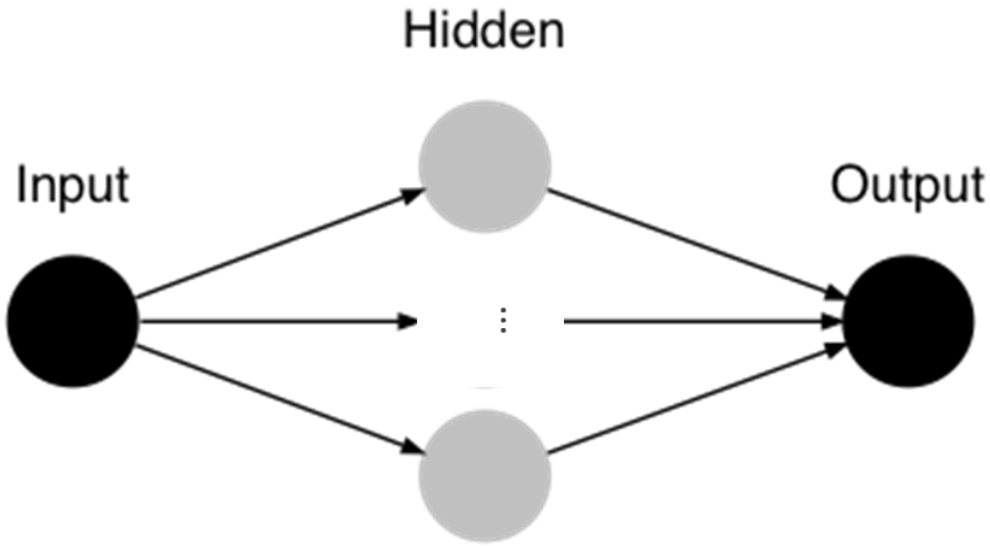


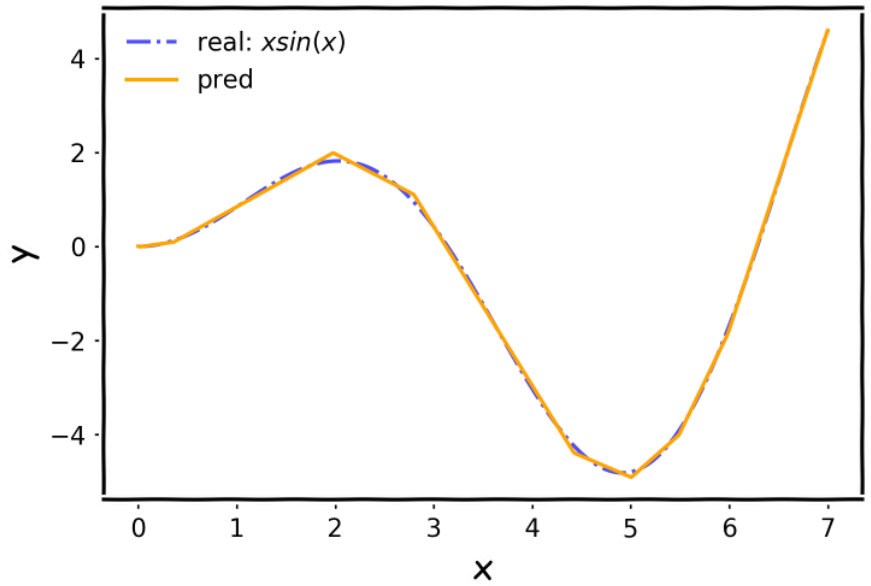
Neural Networks (Architecture / Non-linearity)



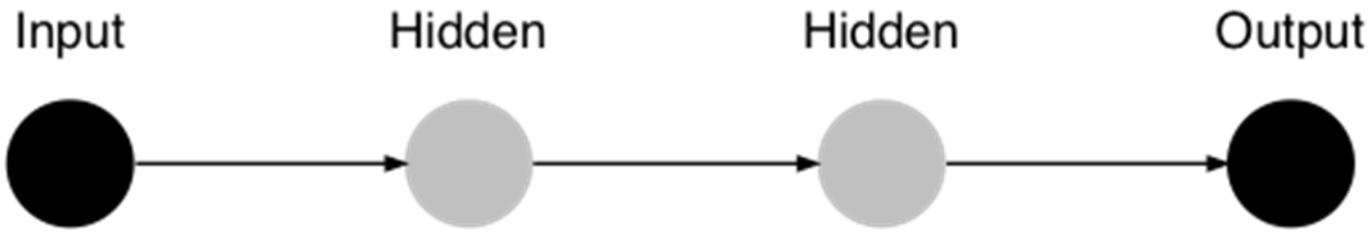


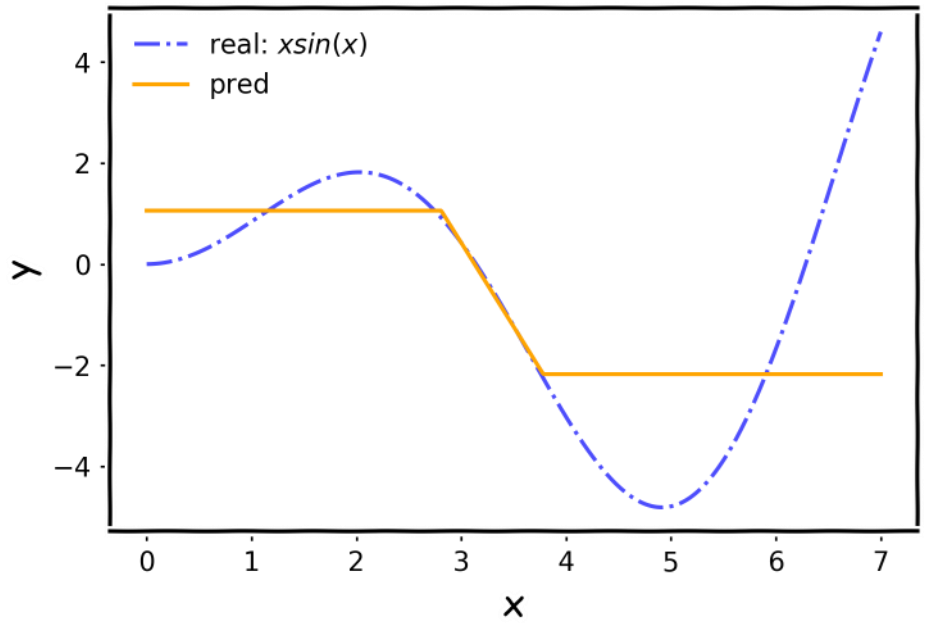
Neural Networks (Architecture / Non-linearity)



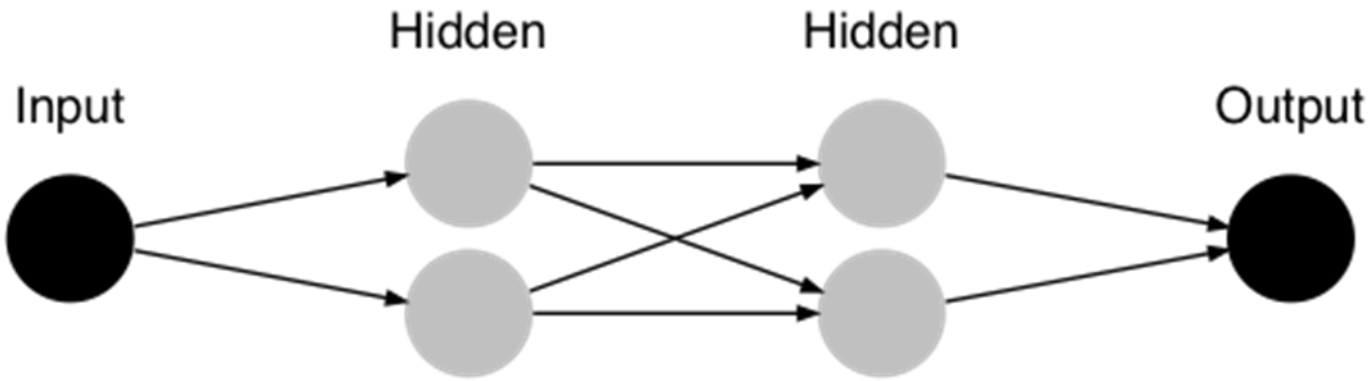


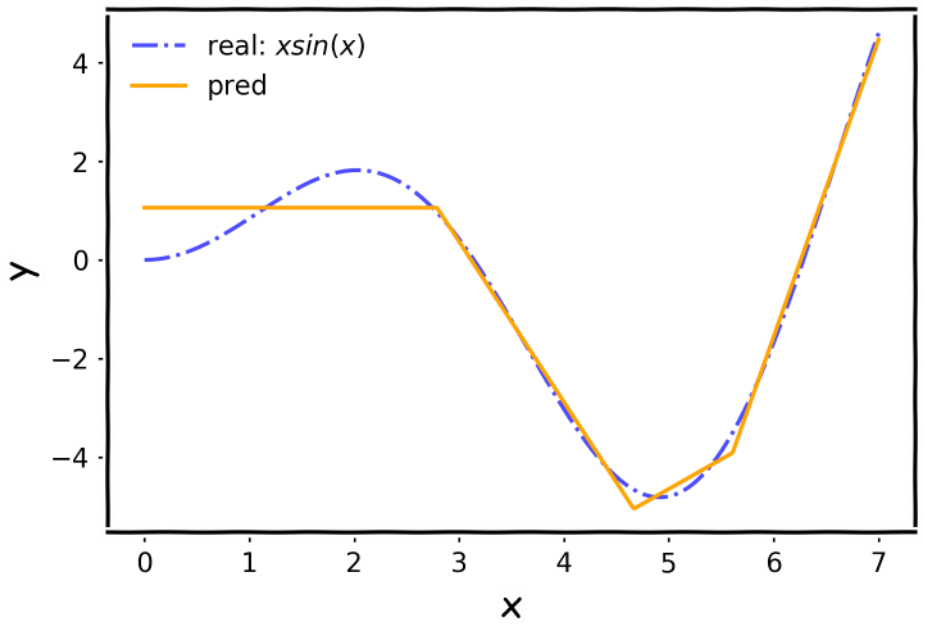
Neural Networks (Architecture / Non-linearity)



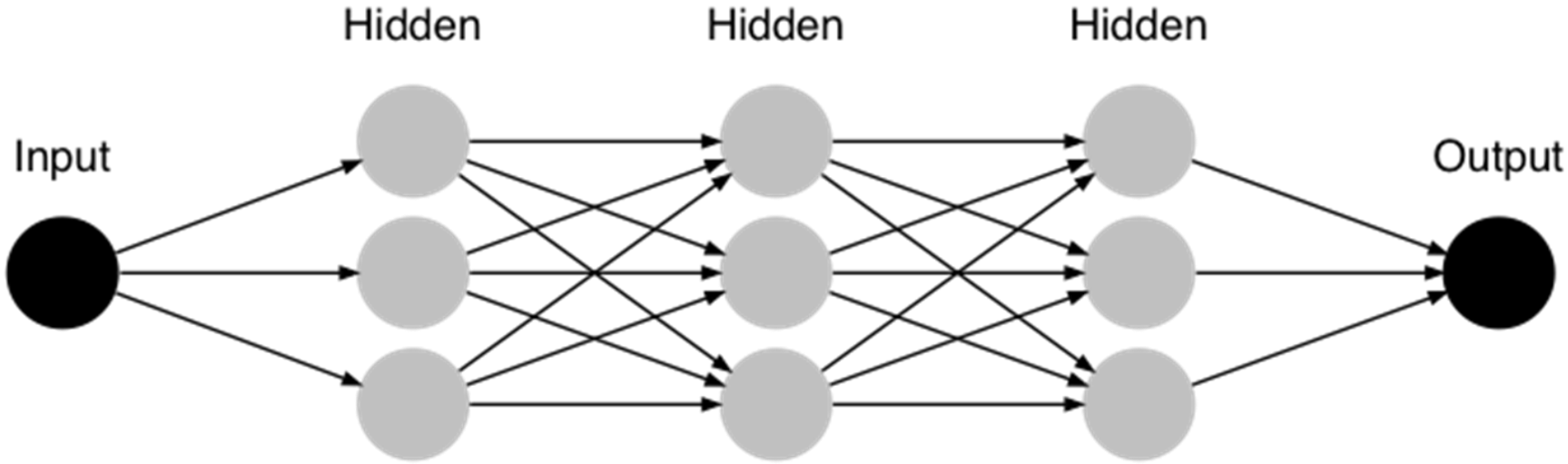


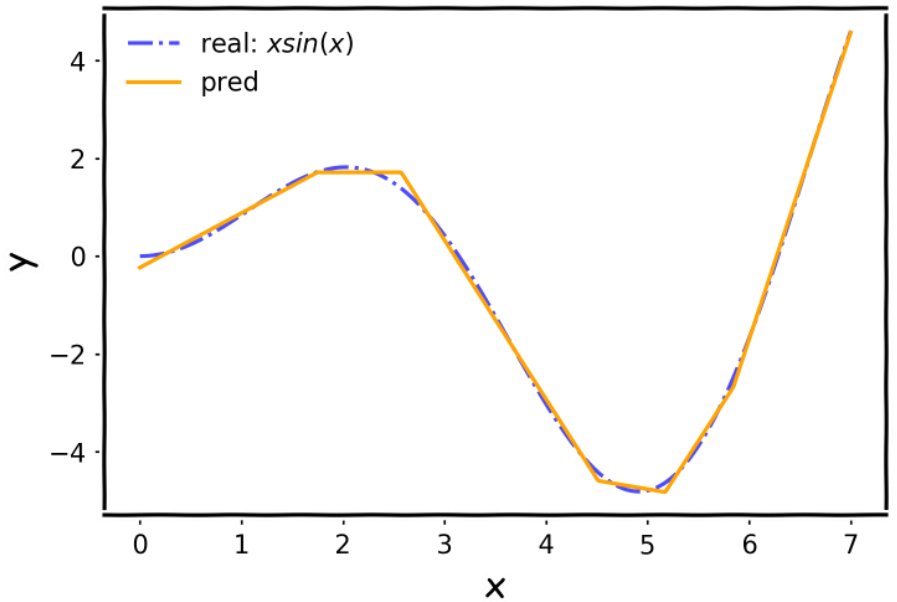
Neural Networks (Architecture / Non-linearity)





Neural Networks (Architecture / Non-linearity)





L07.1

Supervised 6

Neural Networks

=

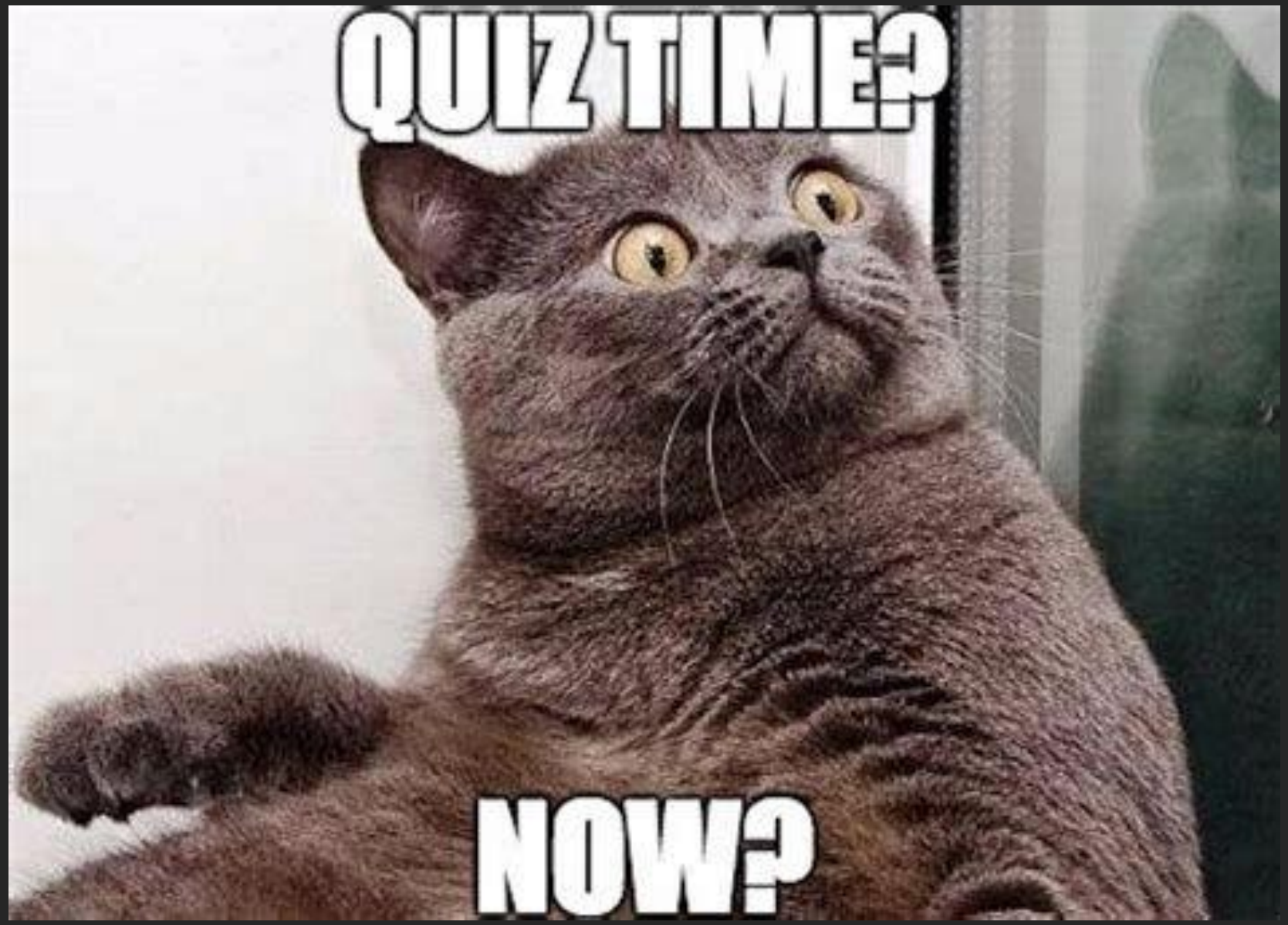
L07.2

Simulations

Rhino

ClimateStudio

Grasshopper


QUIZ TIME?
NOW?